

1. Introduction

1.1. Overview

In the current era the internet has become one of the most important daily use for most individuals, the use differ from person to person but what we all share in our use that we have a private information that we want to keep and no one to know about. But unfortunately cyber attackers and cybercriminals always love to ruin the fun. An attacker can have access to your data black mail you or even worst than that. Though there is always one type of malware that stood out that can have the most reputational and financial loss for individuals as well as gig companies.

In this context, Ransomware could be defined as malicious software used by cybercriminals to encrypt the data of victims, thereby denying access to such data during the period of encryption. Upon encryption, the attackers then demand money in return for the decryption key to access the data again. This means that the victims have no other option but to pay the money and then wait for the attacker to provide the decryption key to access the data. At times, the attackers also offer to provide the victims with an alternative payment for the data, which is often very expensive for the victims.

Therefore, the early detection of Ransomware is important for the mitigation of the associated risks. Early detection of Ransomware helps prevent data loss, financial costs, and the disruption of the attack process, which limits the extent of the resulting financial costs. For instance, the WannaCry Ransomware attack has been estimated to have caused damages of about 4 billion dollars globally. To address the problems associated with the detection of Ransomware, the project will employ machine learning methodologies. For instance, the machine learning approach will employ behavioral data, including process resource usage, entropy patterns of accessed data, and system calls (syscall), to differentiate between standard and Ransomware processes.

1.2. Problem Statement

Ransomware attacks consider one of the most security threads that impose large security. Mostly focuses more on large enterprise via phishing attacks. A main attack vector is phishing attack, where adversaries trick employees into downloading malicious attachments making them think as its a legitimate file such as Word documents, Excel sheets, or PDFs. Once executed, this file may without notice deploy Ransomware, encrypting critical data or even all the data and ask a ransom for decryption. (Kapoor et al., 2022)

Old detection mechanisms often rely on static signatures or rule based systems, which struggle to detect newly emerging, obfuscated, or polymorphic Ransomware variants as mentioned in (Svet et al., 2025). These limitations require more smart and adaptive solutions that can identifying threats based on behavior factors rather than static characteristics. In this context, artificial intelligence, particularly machine learning, shows a powerful results in the recent years to protect data and detect novel Ransomware activity through behavioral analysis.

Furthermore, human error still a main threat , as employees may unintentionally trigger infections through phishing or social engineering attacks which is most likely due to ignorance of the danger. Even with antivirus software and email filters in place, smart cyber groups can bypass conventional defenses using zero-day exploits and evasive techniques. (Cen et al., 2024)

One big issue that existing in the current ear of security is **Partial Encryption** (McIntosh et al., 2019). In normal Ransomware behavior system that monitor process resource usage or simple static feature can catch the Ransomware. Nevertheless **Partial Encryption** Ransomwares operate on somehow different base they do not consume a lot of resources because they only encrypt a part of the file just to corrupt it and that does not consume a much resources. So this consider a major issue regarding Ransomwares.

Finally having all of these information in centralized dashboard such **Splunk** would be grate in which that the person reasonable for monitoring alerts does not need to jump from dashboard to dashboard to check each.

This shows the urgent need for a better, AI-driven Ransomware detection system that can operating in real time. The system shall be able to Utilize Machine Learning using dynamic features hoping to stop the Ransomware before its too late. The following features are required in such system:

1. **Monitor File Activity via system calls:** Track file access via system calls utilizing known Ransomware patterns. As well as **Entropy** monitor for the written data.
2. **Trigger Real-Time Alerts and AI-Based Response:** The confidence level of the classifier can be used to ascertain the level of threat and trigger the alert accordingly. This can also be used for sending the alerts to the SIEM platforms for better handling for the SOC team.
3. **Static Analysis:** Though **Static Analysis** can be something from the past having it well still be able to detect **known** Ransomwares

1.3. Project Objectives

1.3.1. Minimize Ransomware Impact Through Early Detection and Automated Response

The principal objective is to detect Ransomware at the first phase of execution, or even better, before encryption occurs, but that is very hard. By using real-time monitoring with ML-based analysis, the system can respond accordingly to minimize potential damage.

1.3.2. Apply Both Static and Dynamic Analysis for Better Detection

To be able to detect most kind of Ransomwares utilizing both **Static / Dynamic** analysis can help us to achieve that.

Static Analysis:

- The static analysis will primarily concentrate on obtaining all the processes and access their executable source binary by using “VirusTotal”.

Dynamic Analysis:

- Observe the process in real-time to obtain information, such as file I/O, system resource consumption, and entropy values.
- Track system calls and file access sequences as well as abnormal runtime activity.
- Detect anomalies through behavioral profiling, with features aggregated and passed to a machine learning model for classification.

1.3.3. Detect and Mitigate Major Ransomware Categories

The system is designed to be able to identify the following kinds of Ransomware:

- **Full-File Encryption Ransomware:** This type of Ransomware will encrypt all the files using **cryptographic** algorithms such AES, RSA, etc...
- **Partial Encryption Ransomware:** This type of Ransomware encrypts

1.3.4. Integrate with SIEM Platforms (e.g., Splunk)

To support centralized threat analysis and response workflows, the system supports integration with SIEM solutions. This includes:

- Sending alerts and detection logs as well as severity scores to platforms such as Splunk will be much better for centralization than having an interface for it.

1.3.5. Leverage Machine Learning for Behavioral Detection

Machine learning is integrated into the core detection system to enable the system for better and efficient threat recognition.

This includes:

- Using labeled behavioral data taken from system calls, process metrics, and entropy patterns, a supervised model called Random Forest is trained.
- Embedding the trained model in the detection engine for better real-time detection.
- By using training the module to give percentage as prediction this will help in the severity that should be sent.

1.4. Project Scope

1.4.1. In Scope

Detection and Analysis of Ransomware types:

- **Traditional known Ransomware:** Encrypts full files using symmetric or asymmetric encryption and then ask for a ransom from the victim to get the decryption key. Which can be a form of crypto currency.
- **Partial Encryption Ransomware:** Encrypts parts of the file, making it loss data . This is done in order to reduce the entropy changes and avoid the traditional detection techniques.

Static and Dynamic Malware Analysis:

- **Static Analysis:** Utilize the static factors, i.e., the signature of the file and compare it with the known signature.
- **Dynamic Analysis:** This analysis is based on the monitoring of the behavior of the malware, i.e., the access pattern of the file, the variation in the entropy, and the abnormal behavior of the malware.

Process Monitoring and System Call Tracing:

- Monitors running processes and traces their system call sequences to detect low-level behavioral anomalies associated with Ransomware.

Integration with SIEM Platforms:

- Allow for alert forwarding and log export to external Security Information and Event Management (SIEM) platforms. The system will support few of the most known such Splunk. And in future we allow to integrate any SIEM.

Machine Learning-Based Behavioral Detection:

- Utilizes supervised machine learning models to classify process behavior using aggregated features such as CPU usage, entropy shifts, and system call sequences.
- Adding the trained model in the detection system itself to provide real-time classification and severity scoring for better severity classification.

1.4.2. Out of Scope

- Detection or prevention of phishing attacks and campaigns.
- Detection of malware other than Ransomware.
- Development of decryption or data recovery tools for encrypted files.
- Implementation for Windows operating system.

1.4.3. Project Limitations

1. False Positives

- Machine learning-based detection systems can result in wrong classification such as classifying normal processes as malicious due to behavior that consider normal for that process.
- If that happen this may result in annoying alerts that is false and worst to disturb the workflow.
- To avoid this the model should be frequently updated with updated datasets.

These should be employed to degrade unnecessary alerts. As well as setting the checking duration to time that suit the system.

2. Detection Evasion Techniques

- Modern Ransomware normally uses very smart methods of evasion such as code obfuscation, polymorphism, and fileless execution.
- All these methods are meant to evade static signatures and circumvent traditional antivirus as well as smart systems that depend on static rules as detection factors.
- While this project apply static and dynamic analysis, no solution is 100% safe from well crafted zero day or stealth Ransomware and there never will.

3. Resource Consumption

- The real-time monitoring features such as entropy analysis and system call tracing can add a very high overhead to the system.
- This is especially important in low-resource environments, or production environments where the degradation of performance is not acceptable.
- Service priority, analysis providers, and logging options that could potentially maximize the accuracy of detection while minimizing system responsiveness.

1.5. An Overview of methodology

This section covers an overview of the methodology used from collecting data to system design and architecture. To achieve that the following factors has to be taken into considerations:

1.5.1. Dataset Construction

Using two of the most well known dataset to train the module. (Willers & Philippart, 2024) & (Hirano & Kobayashi, 2025) both of these dataset was used to train the module more details about the dataset and what columns has been used from each will be discussed more in chapter 4,5

1.5.2. Feature Extraction and Aggregation

For each process that invoke a hooked system call the following information will be collected:

- Process executable name
- File entropy changes over recent access events
- Recent system call sequences

After collecting these information a module called `machine_learning_attrb` will be responsible to scale the data and arrange them for the module to be able to used them.

1.5.3. Model Training and Validation

The module will be trained using **Random Forest** ("Random Forest Algorithm Overview," 2024) algorithm one of the most well known algorithms for machine learning modules that is very efficient and effective.

To validate the module we can use the known ML mtrics including F1-score, recall, accuracy, AUC. All of these will help to make sure that the module is train properly

1.5.4. Architecture Design of System

The system mainly using Rust for the user space as well as utilising libraries like tokio (for multi-threading and asynchronous) and aya to support eBPF. And C for the kernel space part of the code.

- **Process Monitor:** Record interesting features such the number of open syscall in sec.
- **Entropy Monitor:** Whenever a process calling write syscall. The Entropy will be calculated for the written data.
- **Signature Verifier:** Compares executables against an established signature database.
- **MLClassifier:** Process classification in real-time according to behavioral characteristics.
- **AlertManager:** It warns and logs any detection of activity that is in any way suspicious.

1.5.5. Implementation

All the modules have been implemented using the Rust language for better performance and security. It uses safe multi-thread communication using a very well known library named **Tokio** for the purpose of data exchange between the monitoring modules and the classifier. Alerts are generated in real-time based on the results of signature matches or ML predictions.

1.5.6. Testing and Evaluation

The system will be tested in a controlled setup against benign and Ransomware samples. The testing stages shall be, but not limited to:

- **Functional Testing:** Making sure that the functionality of the system to be serve as promised
- **Integration Testing:** Making sure that the all the system parts can communicate with each other with no problems.
- **System Evaluations:** Testing system performance and overhead that it cause when run on a system.

The model progressed through iterative updates driven by false positives/negatives during validation.

1.6. Target Audience

The system shall target wide range or users from small to medium enterprises, as well as Large enterprises. And surely interested individual whom love to make sure their systems are safe from such attacks.

- **Individual Users**
 - **Tech-Savvy Individuals:** People whom interested in Linux and using it on regular-base can use the system to increase the protection against Ransomware attacks
 - **Remote Workers and Home Office Users:** Individuals whom works from house and uses company devices.
- **Small to Medium Enterprises (SMEs)**
 - **Small to medium Businesses:** Businesses that are consider startups or medium enterprises that can have a single office or multiple.
- **Large Enterprises and Organizations**
 - **Corporate Environments:** These are enterprises that need central monitoring and alerting and need to integrate with other existing SIEM solutions like Splunk.

1.7. Conclusion

After these information it safe to say that this research paper as well the project implementation will provide a gaudiness on implementing low level detection system and overview on Linux and how it deal with such operations.

Also including hybrid approach integrating both static analysis as well as dynamic analysis doing the best to achieve the maximum coverage that can be achieve to protect Linux systems.

While also providing centralize alerting system that does not introduce new interface but it integrate with what the industry currently rely on

2. Literature Review

2.1. Overview

This section will mainly focus on the previous researches and what has been found and identify. By utilizing these existing research papers we can get an idea on the existing solutions and how to benefit from them in the current implementation of the project.

2.2. Ransomware Detection, Avoidance, and Mitigation Scheme: A Review and Future Directions

2.2.1. Paper Overview

The paper by (Kapoor et al., 2022) shows an overview look at Ransomware operations and ways to counter them. It shows the common and known methods for detecting Ransomware while it is running and shows plans to help make the system defenses stronger. Moreover, the paper covers the whole Ransomware lifecycle from initial drop methods like phishing to prevention techniques such as patch management, providing an overview on how Ransoms actually function.

2.2.2. Ransomware Operation

The paper discusses the operational phases of a Ransomware attack in a structured manner that makes it easy to understand and follow. These stages include:

- **Infection:** The initial phase in which the Ransomware cracks the target system's external protections, often through phishing emails or malicious downloads.
- **Encryption/Locking:** In this phase of the Ransomware attack, the Ransomware will encrypt important files or lock the user out of the system or even encrypt everything on the machine, making the data inaccessible.
- **Ransom Demand:** The attacker communicates with the victim instructing them to visit usually websites that host on the **Dark web** usually shows .onion extension. They usually demand a crypto currency making it very hard to trace.

-**Outcome:** The victim has the choice to pay or not. Unfortunately, in many cases payment is the only option presented, and even after payment they may still keep your data and possibly sell them.

2.2.3. Techniques in the DAM Framework

The paper discusses the DAM which stand for **Detection, Analysis and Mitigation** framework, which provides a structured approach to identifying and responding to Ransomware. It indicate the importance of integrating both **static** and **dynamic analysis** techniques to enable the system to protect and detect as early as possible consider all the possibilities in mind.

- **Static Analysis:** Static analysis usually covers as it name implies static features from executable signature to function used and dynamic libraries used in the executable.
- **Dynamic Analysis:** Dynamic analysis on the other hand take a lot into consideration the meaning of it has change recently due to the use of AI/ML. Nevertheless its original meaning refers to the dynamic features that can be change overtime while the program is executing like CPU/RAM usage, Disk access, System calls access and more.

2.2.4. Ransomware Avoidance Techniques

The paper gives general best practices for avoiding Ransomware attacks. While some techniques are consider very specific though still these practices can and will help to protect from Ransomware attacks from the roots.

- Regularly apply system patches and updates to avoid known vulnerability.
- Avoid opening email attachments or clicking links from unknown or untrusted sources they usually consider scamming or phishing attempts.
- **Least Privileges Principle** is very helpful limiting access to important files to an authorized users.

2.2.5. Mitigation Strategies

While the paper discusses multiple tools, this review focuses on the main mitigation strategies other than specific implementations:

- **Prevention:** Techniques such as IP/domain blacklists and traffic inspection to block command-and-control (C2) communications before malware executes.
- **Detection:** Use of behavioral monitoring as well as entropy-based detection and ML-powered classifiers to identify Ransomware activity in real time can help in early detection and avoid losses.
- **Containment:** Isolating infected systems using network segmentation to disable the Ransomware from replicate it self among other machines.
- **Recovery:** Having a backup or snapshots is very important to be able to recover with the minimal losses.
- **Reverse Engineering:** Applying both static and dynamic analysis to extract encryption keys or understand Ransomware logic can be very useful for developing decryption tools or signatures that will help in the future.

2.3. A Comprehensive Review on Malware Detection Approaches

This paper by (Aslan & Samet, 2020) presents a detailed overview on how Malware generally are detected, discussing the strengths and limitations of current techniques while also covering novel methods that may shape future detection in the context of Ransomware.

2.3.1. Problem

The paper discusses the challenges involved in malware detection by considering the general methodologies used for different kinds of malware. While the paper is not limited to Ransomware pedagogy, the techniques for malware detection discussed here can be applicable for ransomware as well.

2.3.2. Paper Overview

The authors begin by outlining the evolving malware landscape and then categorize common detection methods.

- **Malware Evolution:** Old malware variants were often very simple and relatively easy to detect. However, modern threats utilize advanced obfuscation techniques and very complicated and novel ways to get around the existing protection systems.
- **Current Malware Landscape:** The paper highlight the growing diversity of malware threats, including ones targeting IoT devices, mobile platforms, and cloud environments, shows that Malware is and will remains a very hard attack vector to stop and completely protect from.
- **Malware Detection Methods:** The paper covers both techniques static as well as dynamic analysis as the two primary approaches in malware detection.

2.3.3. Malware Detection Approaches

The detection approaches are arranged into two main strategies signature/behavior analysis. These align with the static/dynamic analysis both are explored in depth as follow:

- **Signature-Based Detection**

1. **Overview:** Involves creating unique identifiers usually called signatures for known malware, which are then matched against signatures that usually generated via SHA256/MD5 hashes of the malware binary.
2. **Techniques for Signature Generation:**
 - **String Scanning:** Finding a static values usually at the beginning of the file.
 - **Top-and-Tail Scanning:** Focuses on the beginning and end of files, where modifications usually happen in there.
 - **Entry Point Scanning:** Scans bytes at the beginning and the end of the binary searching for possible modifications
 - **Integrity Checking:** Uses cryptographic hashes for example SHA-256 or MD5 to detect unauthorized changes in executable files.
3. **Challenges:**
 - Useless against new or obfuscated or even slightly modified malware variants.

- **Behavioral-Based Detection**

1. **Overview:** Such a strategy also emphasizes the importance of studying program behavior within a system. This enable the potential to reveal unknown malware by recognizing behavioral patterns that learned from previous Ransomwares that have been seen by the running process which can be classified as malware, even when it has been hidden using code obfuscation techniques.

2. **Behavior Detection Techniques:**

- **Sandbox Execution:** Sandboxes enable running Malware safely to observe what they do and analyzes based on that.
- **System Call Monitoring:** Tacking the requests made from the program to the OS usually contains patterns that can be detected.
- **Network Activity Analysis:** Monitor network traffic to find find suspicious traffic.
- **Process Monitoring:** Tracking process creation and resource usage.

3. **Evaluation and Challenges:**

- **Behavior Definition:** Its hard to be able to tell exactly which type of Malware is running except if the behavior detection is designed to monitor single type or Malware.
- **Sandbox Evasion:** Some malware is designed to detect virtualised or sandboxed environments, leading to false negatives.
- **Obfuscation:** Behavioral changes may be subtle or masked entirely through advanced techniques.

2.3.4. Conclusion

The paper also looks at malware detection in newer areas like cloud and IoT systems, but our review focuses on endpoints, especially Linux systems. The comparison between signature-based and behavior-based methods shows that combining both approaches is effective. By utilizing behavioral analysis along with machine learning, the system will be able to detect known threats and also adapt to new and evolving Ransomware.

2.4. A Survey of Crypto Ransomware Attack Detection Methodologies: An Evolving Outlook

2.4.1. Paper Overview

The following paper by (Alqahtani & Sheldon, 2022) shows different methodologies for detecting crypto Ransomware, with a focus on new Ransomware variants and the limitations of existing research.

The study shows how older detection methods often are not able to detect the full scope of Ransomware behavior, particularly in its early execution stages and discuss better solutions that are able to match with the new generations of Ransomwares

2.4.2. Crypto Ransomware Detection Approaches

The paper arranges Ransomware detection into two main approaches **data centric** as well as **process centric** discussing each one of them pros and cons

1. Data Centric Approaches: These methods usually focus on analyzing the impact of Ransomware on data itself rather than monitoring the Ransomware's behavior directly. In other words in this type of analysis the focus is on the data change not the Ransomware. Common techniques include:

- **Entropy Analysis:** Encrypting files will result in a high entropy meaning if the data change from normal entropy to a high entropy this may indicate encryption attempts on the data.
- **Decoy (Honey) Files:** These are files that trick the Ransomware into believing that they are important and they are just put as a bait for it to analyze.

Limitations:

- Not all operations that cause a high level of entropy often mean an encryption attempt. Zipping files will generate a high level of entropy as well.
- Advanced Ransomware may encrypt real user files before touching decoys.
- Entropy-based methods may be a bad choice if files are already compressed or encrypted.
- Sophisticated malware may avoid or bypass decoys entirely by accessing files through non-standard methods.

2. Process-Centric Approaches: This method is the opposite of the previous one it focus more on the Ransomware itself and its behavior than the data.

- **Predefined Event Detection:** This part focus on the events that the Ransomware will use for example encryption we can hook the encryption function when it called an event will happen enabling the protection system to tell and collect data.
- **Behavioral Pattern Analysis:** This focus more the resources patterns like disk access and system calls to find a pattent that can be match with Ransomware behavior.

The paper also separate two major techniques under this category:

- **Event-Based Detection:** Tracks known Ransomware specific actions like:
 - Creation of encryption keys
 - Checks if the binary is using libraries like `Openssl`
 - Communication with command and control (C2) servers
 - Use of domain generation algorithms DGA which helps in hiding the C2 server domain.

Limitations:

- Requires prior knowledge of Ransomware behavior which is very different from Ransomware to another.
- Normal programs may use similar APIs leading to false positives.
- Ransomware with it getting more and more smarter its easy for it to generate false information to mislead the protection system.

- **Machine Learning-Based Detection:** Basically train the module on a features that can be collected from the system and enable the ML to predict the likelihood for a process to be a Ransomware

Advantages:

- Generalizes better to unseen Ransomware variants.
- Can detect subtle patterns across multiple features system calls, entropy, resource usage.

Challenges:

- Requires high quality training data which is the case for Linux Ransomwares it lack the open source data that can be used to train the module.

2.4.3. Limitations of Existing Research in Early Detection

The article tells that the early detection of Ransomware is still the most difficult part of Ransomware protection. Crypto Ransomware may behave like normal applications during the pre encryption stage, making it hard to detect solely based on standard rules. Most current models have the following problems:

- Modules can't tell what is happening before encryption is started meaning its very difficult to detect it before it starts.

This issue highlight the need for smart monitoring and dynamic detection approaches, such as real time machine learning models that assess aggregated features such as system calls, entropy changes, and resource consumption, to enhance early detection accuracy.

2.5. Ransomware Detection Using LSTM Networks and File Entropy Analysis: A Sequence-Based Approach

2.5.1. Paper Overview

This paper by (Kiyol et al., 2024) covers many aspects, but we will focus on the entropy part as it contributes to our research.

2.5.2. Entropy-Based Ransomware Detection

The paper highlights an entropy-based approach to detect Ransomware attacks, focusing on how each method can be used in identifying early characteristics of encryption activity. Entropy in this context refers to the measure of randomness within a file's content. When Ransomware begins to encrypt files, it would alter their structure completely increasing the level of entropy of that file. This change in file entropy can be a likelihood of a Ransomware activity.

By continuously monitoring the entropy levels of files or directories the system can detect Ransomware like behavior increases which will increase entropy levels, which might be a clear indication of the ongoing encryption process. The mathematical principle behind entropy, usually calculated using the formula for entropy as presented by Shannon, can be used to determine the changes in entropy levels. This makes the process precise and automated.

However, although the entropy technique can be very effective, though it might sometimes lead to false positives, especially if programs such as compression programs can generate a high level of entropy, causing the entropy levels to rise. Regardless of the challenges associated with the entropy technique, it can be very effective if used with other techniques for monitoring the Ransomware process. The formula for calculating the entropy is as follows:

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

Shannon Entropy of a discrete random variable

Where: $P(x_i)$ is the probability of the i -th byte value in the file. And $N = 255$. The total number of possible values is represented by the range from 0x00 to 0xFF.

The logarithm is taken with a base of 2, as the domain is binary and information is quantified in terms of bits.

The entropy represents the probability distribution of the byte values present in the file. A high value of entropy represents a high level of randomness, which is often related to data that has been encrypted or obfuscated.

2.5.3. Conclusion

This paper provides an extensive overview of the concept of entropy and its application in ransomware detection. It discusses the theoretical background of the concept of entropy as a measure of randomness in digital files. It highlights the importance of the concept of entropy in the detection of ransomware attacks. By analyzing changes in the values of entropy, especially the sharp rise that is normally seen in ransomware attacks, it is possible to use it as an indicator of ransomware attacks. Therefore, it plays an important part in the general framework of file-based ransomware analysis.

2.6. Real-Time Ransomware Detection Using Behavioral Entropy Analysis for Secure File Systems

2.6.1. Overview

This paper proposed by (Breus et al., 2024) focus on improving and evaluating the use of entropy as a real time ransomware detection technique, focusing on how an entropy can be effectively used in detection systems that aims to counter ransomware attacks.

2.6.2. Behavioral-Based Detection

The author Breus et al. states clearly that dynamic analysis has the potential to capture Ransomware before significant damage occurs. Which is totally correct though the issue lies on can we prevent it before any damage can happen ??

Two primary methods are discussed:

1. **Heuristic-Based:** This approach is very similar to dynamic analysis and relies on certain behavioral feature Ransomware has to detect its activity.
2. **Signature-Based:** This method is a reliable and used alot in traditional detection system. Though in the current era of Ransomware they suffer from the fact that they relay on static features only any change in them will lead to no detection.

2.6.3. Advancements in Entropy-Based Analysis

This section covers entropy based detection in more detail. The main points include:

- **What is Entropy-Based Detection:** Entropy as discussed in Section 2.5 is just a formula that clauclate the randomness of the bytes in a file. Encryption attempts will cause that number to spikes.
- **How It Works:** The system will watch real-time file changes and will flag the files that generate that high level of entropy.
- **Advantages:** Entropy-based detection still effective even when signature/behavioral methods fail to detect. It can detect advanced obfuscated Ransomware and adapt to new version of Ransomwares.
- **Technical Aspect:** The strong points of this method lies in its how it utilize math that is hardly manipulated by the Ransomware because the core idea of Ransomware is to encrypt, if it does not encrypt to avoid generating high level of entropy then there is no damage.

- **Hybrid Approach:** Combining dynamic analysis with entropy-based detection help in detecting more advance Ransomwares.
- **Challenges:** The primary limitation is the computational resources required to support real-time monitoring as well as Particularly encryption Ransomwares.

2.6.4. Behavioral Entropy Metrics

The Behavioral Entropy Detection Protocol (BEDP) utilizes advanced entropy metrics to detect Ransomware by analyzing changes in file randomness. Using Shannon entropy and differential entropy, it is possible to detect sudden spikes in file encryption, while file comparisons are conducted using variance-based methods. These methods compare files before and after modifications. In addition, thresholds are established to identify files with high entropy. BEDP also analyzes file access frequencies and utilizes a temporal analysis approach to identify normal system activity from malicious activity, as Ransomware attempts to mimic normal system activity.

2.6.5. Detection Algorithm

BEDP uses a detection algorithm that continuously monitors the fluctuation of real-time entropy and the behavior of the files. It determines the baseline of the entropy of the files and then detects any sudden spikes. It then uses a machine learning approach to ascertain whether the detected behavior is consistent with the previously defined ransomware behavior.

2.6.6. File System Monitoring Setup

The author Breus et al. Will take a directories to monitor from the user and will start a thread for each to monitor. The system tracks file events and calculates entropy values after each modification. Then if it find that a spikes occurs in the entropy before and after access it will generate an alert other wise the entropy will be updated for the file.

2.6.7. Results

The experimental results shows the BEDP effectiveness in detecting Ransomware with high accuracy and precision. The primary downside is that the system use a lot of resources. Other than that the key performance metrics such as detection rate, processing speed, and error margins—were evaluated, confirming the system's reliability across multiple test scenarios.

2.7. Random Forest Algorithm Overview

This research paper (“Random Forest Algorithm Overview,” 2024) provides an overall idea of the **Random Forest (RF)** algorithm, a widely used machine learning technique for classification and regression tasks. The paper discusses the basic concepts, operational mechanisms, advantages, challenges, and applications of Random Forests and more.

2.7.1. Introduction to Random Forests

- Random Forest is described as an important machine learning model used for data mining, which has been proven to be effective.
- It is described as an approach that uses many decision trees on random subsets of data and then combines the results to produce the desired predictions.
- It is noted that the quality of the training data is crucial, and the role of system performance data is emphasized.

2.7.2. Random Forest Algorithm

- The Random Forest algorithm is described as an ensemble method of decision trees to increase the accuracy of the predictions and avoid overfitting.
- The algorithm makes use of bootstrap sampling with random feature selection to create decision trees.
- The algorithm is computationally fast and is able to handle high-dimensional features. It is used for both classification and regression.

2.7.3. Benefits and Challenges

- **Benefits:** High accuracy and resistance to overfitting, missing data does not effect the algorithm too much as well as the ability to see and evaluate the importance of each feature used to train the module.
- **Challenges:** If the dataset is large the training time will take too long due to it egenrating many trees

2.7.4. Technical Features

The article highlights several important technical aspects:

- **Bootstrap Aggregation (Bagging):** This is the construction of multiple training data sets using sampling with replacement.
- **Feature Selection:** This is the selection of features based on their corresponding feature importance scores.

- **Out-of-Bag (OOB) Estimates:** This is internal validating using data not used for training.
- **Variable Importance Measures (VIM):** These are used to measure the feature contribution to model performance.
- **Proximity Measures:** These are used to evaluate data points and are used in genetics.
- **Handling Missing Data:** proximity matrices and k-nearest neighbors which is utilized by this algorithm helps in padding the missing data.

2.7.5. Optimization

- The paper discusses more things about **Random Forest** such hyperparameter tuning and cross-validation and how that can help in technique optimizing the model performance.

2.7.6. Results and Conclusions

- Random Forests are considered one of the most used algorithm due to their accuracy, robustness, ease of use, and ability to handle different data types and complex problem.
- The algorithm is considered easy to use and useful for both classification and regression tasks.
- The paper summarise that **Random Forest** algorithm is one of the most powerful and adaptable algorithms for predictions and analytics across for wide range of applications and industries.

2.8. A Novel Static Analysis Approach Using System Calls for Linux IoT Malware Detection

2.8.1. Overview

This paper by (Manoharan & poonkuzhali.S, 2020) discusses the malware for Linux-based IoT devices. It proposes a new technique for analyzing malware statically without executing the malware. It analyzes system calls and library syscall wrappers for the ARM Linux ELF binaries. Its not the same as dynamic analysis which require the malware to be executing while it analyzes it. It analyzed 1,117 malware and 1,214 benign files. It ranked system calls based on security risks and patterns used by Malware authors. It created a dataset to find whether it could differentiate between malicious binaries and benign ones.

2.8.2. Solution

The proposed technique covers multiple fundamental steps:

- **Syscall Extraction and Categorization:** The syscalls and library syscall wrappers are extracted using the radare2 reverse engineering platform. The syscalls are arranged into ten different functionality groups FileSystem, Process, Network, etc... and are given security risk rankings High, Medium, Low.
- **Feature Engineering:** Additional statistical features are covered too in the data set. These include the total syscalls made, the unique syscalls made, frequency of syscalls in each category, percentage of high-risk syscalls, entropy of the syscall sequence, and weighted risk rankings of syscalls.
- **Statistical Validation:** The effectiveness of the syscall categorization and risk ranking is validated using chi-squared and Mann-Whitney tests. These tests show that there are significant gap in syscall patterns between malware and normal binaries.
- **Machine Learning Classification:** The data set is evaluated using several machine learning algorithms. These include Logistic Regression, Random Forest, Support Vector Classification, and Multi-Layer Perceptron Neural Network. The algorithms are used for classifying the binaries as malicious or benign using only static syscall analysis.

2.8.3. Results

The results obtained from the study reveal the effectiveness of the static analysis methodology used for malware detection as follows:

- The Random Forest classifier performed well with an F1 score of 96.86% and accuracy of 93.34% as well as precision of 94.71% which make it a good module that can be used in such systems.
- The performance of the other models such as Logistic Regression, SVC, and the MLP neural network, was very good, with high F1 scores above 95%.
- The statistical tests proved the effectiveness of the generated features and risk rankings for distinguishing malware from benign binaries.
- The process avoids the execution of binaries, thus circumventing the risks involved with dynamic analysis, and can be adapted for different hardware architectures.

2.9. Ransomware Detection on Linux Using Machine Learning with Random Forest Algorithm

2.9.1. Overview

The study conducted by (Wu & Chang, 2024) work with the growing problem of Ransomware on Linux-based systems, which are increasingly being compromised due to the popularity of the operating system in critical systems mainly on server because Linux servers represents 70% of the number of servers in the world.

2.9.2. Problem

Ransomware on Linux-based systems discuss a specific problem related to the various distributions of the Linux operating system and the sophisticated evasion techniques used by Ransomware authors to avoid the existing protection systems. Existing methods are often not capable to keep up with the dynamic nature of Ransomware and have high false positive rates or are not sufficiently adaptable caused by the diversity of Ransomwares.

2.9.3. Solution

The authors Wu & Chang propose the application of machine learning-based detection methods utilizing the Random Forest algorithm for detecting Ransomware on Linux systems. The following steps are involved in the study:

- **Dataset Collection:** A dataset from various Linux distributions has been collected including normal and Ransomwre files.
- **Feature Engineering:** Needed features related to the file, system calls, network communications, and processes are extracted.
- **Preprocessing:** Normalizing the data is require, handling missing data, encoding the data, balancing the dataset.
- **Model Training:** The Random Forest model needs to be trained and optimized using the Grid Search and Cross-Validation methods for the best possible results.

2.9.4. The Use of Machine Learning

The Random Forest algorithm was selected due to its robustness and its ability to handle high-dimensional data, and its avoidance of over-fitting problem. The algorithm was trained using the preprocessed discussed earlier data with hyperparameter tuning to ensure optimal performance.

2.9.5. Results

The model shows a very strong performance across key evaluation metrics:

- Precision: 0.93
- Recall: 0.89
- F1-score: 0.91
- Accuracy: 0.94
- AUC (Area Under the ROC Curve): 0.96

According to the researcher Wu & Chang, the results were found to be so much better than the other alternative machine learning models such as Support Vector Machines (SVM), decision trees, and neural networks. These findings validate the superiority of the Random Forest model for detecting Ransomware attacks on Linux platforms.

2.10. Conclusion

Building on what have been discussed so far, I propose a hybrid detection approach that combines all behavioral analysis, entropy-based monitoring, and static analysis as well as machine learning module to form a comprehensive Ransomware detection framework.

This designed enable the system to effectively detect Ransomware ranging from highly obfuscated advanced Ransomware to more simple variants deployed by **script kiddies**.

While static analysis shows a strong advantage of it being able to detect threats before executing it, however it often not able to detect stealthier Ransomware variants specially those designed by Advanced Persistent Threats (APT), as highlighted in those researches (Breus et al., 2024), (Kapoor et al., 2022). On the other hand, both behavioral and entropy-based techniques shows a very promising ability in detecting encryption activities during runtime of the Ransomware.

Thus, a hybrid model balances between static detection as well as behavioral and entropy monitoring, providing layer of defense that can be integrated with other layer forming a well defended system,

It is important to keep in mind, however, that achieving a 100% detection kinda impossible due to that fact Ransomware is being generated everyday with different capacities and features as well as tactics and evasion techniques.

3. Requirements Analysis

3.1. Overview.

In this section, we will conduct an overall analysis of the requirements identified in previous research, along with ideas getting from expert in the field of Cybersecurity. The main objective in the section is to identify features that we can add into the system. Also with observation we can find a lot more details that help us identify more requirements.

3.2. Fact-Finding Techniques

3.2.1. Justification

The main reasons for using the Questionnaire and Observation methodologies are that they are the most appropriate approaches in undertaking this kind of project. This is because, by observing the practices of other people, as well as getting insights from experts, the researcher is able to get guidelines on how to develop an all-inclusive system that can help in mitigating Ransomware attacks.

The Requirements Analysis phase incorporated two methodologies, as described below:

1. **Questionnaire:** A questionnaire was designed to elicit information and views from cybersecurity professionals. The responses provided critical inputs on fundamental characteristics and expectations of the application.
2. **Observation:** In this approach, the usage of the application was analyzed. This was achieved by observing users as well as past research papers. In addition, literature on similar applications was consulted to guide the requirements analysis methodology. This helped in validating the requirements of the application in a practical scenario.

3.2.2. Questionnaire Design

The questionnaire was designed with a mix of both open-ended and close-ended questions. These questions were designed to elicit comprehensive and meaningful responses from cybersecurity professionals.

- **Open-Ended Questions:** These questions allowed participants to share their views, explain their reasoning behind the idea, and give detailed informations based on their practical experiences.
- **Close-Ended Questions:** These questions helped in gathering objective responses from participants about the practicality of the system, its design, and its relevance. These questions had binary responses, with participants answering ‘yes’ or ‘no.’.

The questions evolved from general questions about Ransomware detection to more specific questions about Machine Learning, trust in ML, entropy, and syscall-based detection in Linux.

3.2.3. About Participants

1. Participants had 2-5 years of experience in cybersecurity, shows their expertise in the field.
2. All participants were cybersecurity professionals.
3. Most participants had hands-on experience with malware analysis or incident response.
4. All participants had knowledge of Linux, which was very important since the application is intended for Linux devices.

3.2.4. Feedback on Ransomware Status

1. AI development has improve the quality of phishing attacks, which still the main method used by attackers to deliver the Ransomware.
2. Ninety percent (90%) of participants agreed that signature-based systems are less effective because attackers can easily avoid them with small changes.
3. All the participants (100%) shows trust in using machine learning methods in the detection of Ransomware, regardless of the potential false positives to cause mistrust.
4. All the participants (100%) express the importance of using dynamic indicators which are strong indicators of potential attacks.
5. The majority of the participants were of the view that the most critical phase of the Ransomware life cycle was the delivery.
6. All the participants were in agreement that using a single technique in the detection of Ransomware was poor choice. Meaning utilizing multiple techniques is the way to go.

The feedback from experts on the effect of the use of different Ransomware detection methods clearly showed that no single solution could guarantee complete protection from Ransomware damages. Instead, a combination of signature, dynamic, and ML-based approaches helps in offering the best defense against the changing Ransomware attacks.

This is clearly demonstrate by the comment from one participant who express it in a nutshell said, “It will remain a game of cat and mouse.” This comment explain the ongoing cat and mouse game between Ransomware authors and those who seek to protect computer systems from these threats.

3.2.5. Observation

From the historical analysis and case studies on Ransomware attacks, it is evident that the attacks have caused significant financial and operational impacts on different sectors across the industry landscape. Within the last ten years, some significant Ransomware attacks have highlighted the potential impact of Ransomware if proper security measures are not adopted.

Some of the notable Ransomware attacks include the WannaCry Ransomware attacks, which caused significant disruptions across more than 150 nations around the world. According to the WannaCry study, the Ransomware took advantage of a critical vulnerability in the Windows SMB protocol, known as **EternalBlue**. This vulnerability caused the **WannaCry** Ransomware to spread quickly across the world without the need for user interaction. This illustrates the fact that Ransomware infections do not always come from email phishing attacks.

This brings out the need for the systems to be up-to-date with the latest security patches. Additionally, the inclusion of anti-phishing measures and educating the employees on how to recognize social engineering attempts should be taken seriously. While the technical measures can be effective, the human aspect also plays a significant role in combating ransomware attacks and preventing disasters.

3.2.6. Analysis on Results

Feedback and observation by experts suggest that ransomware is an ever-changing threat with multiple vectors, which cannot be addressed by any single countermeasure. Phishing attacks have remained a major means of initial access, but vulnerability and configuration issues have remained equally significant in this context.

Combining behavioral detection with machine learning, in addition to conventional techniques such as signature-based detection and system resource monitoring, provides a more reliable security solution. Cybersecurity needs to be proactive and dynamic, using techniques such as updates, multiple layers of detection, user education, and real-time behavioral analytics.

3.3. Requirements

3.3.1. Functional Requirements

- Monitor active processes in real time.
- Calculate file entropy during file access operations.
- Trace recent system calls made by monitored processes.
- Compare executable hashes against a signature database.
- Use a trained machine learning model to classify process behavior.
- Trigger alerts if malicious behavior is detected (via signature or ML).
- Export alerts to a SIEM platform (e.g., Splunk).
- Maintain a log file for all analysis, alerts, and predictions.

3.3.2. Non-Functional Requirements

- Minimal CPU and memory usage to avoid interrupting user workflows.
- Compatibility with major Linux distributions (e.g., Ubuntu, Kali).
- Support for multi-threaded execution for scalable performance.
- Machine learning model must maintain detection accuracy above 90%.

3.3.3. User Requirements

- View real-time alerts when suspicious or malicious behavior is detected.
- Configure alert severity levels and whitelist trusted processes.
- Export logs and detection results to external SIEM platforms (e.g., Splunk).
- Access a centralized interface for monitoring and managing detection activities.

4. System Design

4.1. Overview.

This section covers over all design and how each part of the system connects with each other. Also **Machine Learning Module** implementation and dataset will be discussed.

4.2. Rich Picture Diagram

This first section provides a high-level overview showing **Rich Picture Diagram** Figure 1. As well as high-level details that make up the system.

4.2.1. Starting the program

This is a very basic thing that a user can run and start the program via Command Line. Starting the program will launch two main threads one for **Static Analysis** and other for **Dynamic Analysis**.

4.2.2. Dynamic Analysis

In this **thread** dynamic analysis will run, the program going to load and attach all programs registered in the **Ebpf** program. eBPF will be discussed in depth in later sections. And the main shared data structure which is a **Hash Map** a key, value pair of data. The key represents the **PID** process identifier, while the value is a collection of data including number of system calls for system calls such (open, close, delete). After accessing the data in the **Rust** client program data will be organised and scaled to fit the **ML** module. Finally the data will be passed to the **ML** module and if the resulted probability is higher than or equal to the **minimum severity to report** an alert will be sent to **Splunk**. And if it found to be lesser than the **minimum severity to report** the information will be discarded and new iteration will begin.

4.2.3. Static Analysis.

In this **thread** static analysis will run, the idea here is quite simple all process along with their executable will be collected. A **sha256** hash will be generated for all of the executable and will be compared against a known malware database (VirusTotal, n.d.). If the hash were match an alert will be sent with maximum level of severity indicating emergence of the situation, Otherwise the operation will be ignored.

4.2.4. Sending alert.

The operation of sending an alert is rather simple the user will provide the **token** and a **URL** pointing to where **Splunk** is. And using **HTTP** request along with **Json**-

encoded data indicating the incident, once the **HTTP** request is received to **Splunk** then **Splunk** will handle the rest of the operation from showing the alert to the user index it and enable filters and searches.

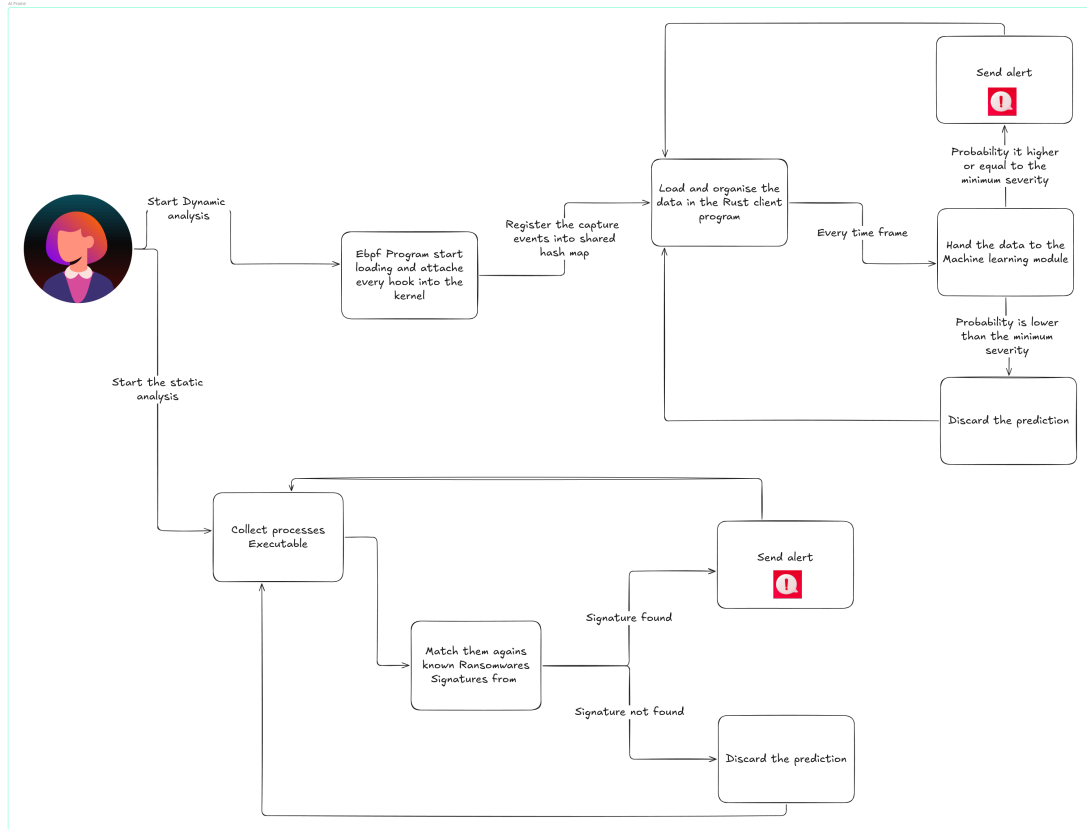


Figure 1: Rich Picture Diagram

4.3. Use Case Diagram.

This use case below Figure 2 demonstrate the system architecture. Basically the user can start the program and these modules shown connect to the user shows the facility the directly connect to the user. Though the main feature is the **machine_learning_attrb** that start the execution of the main program feature.

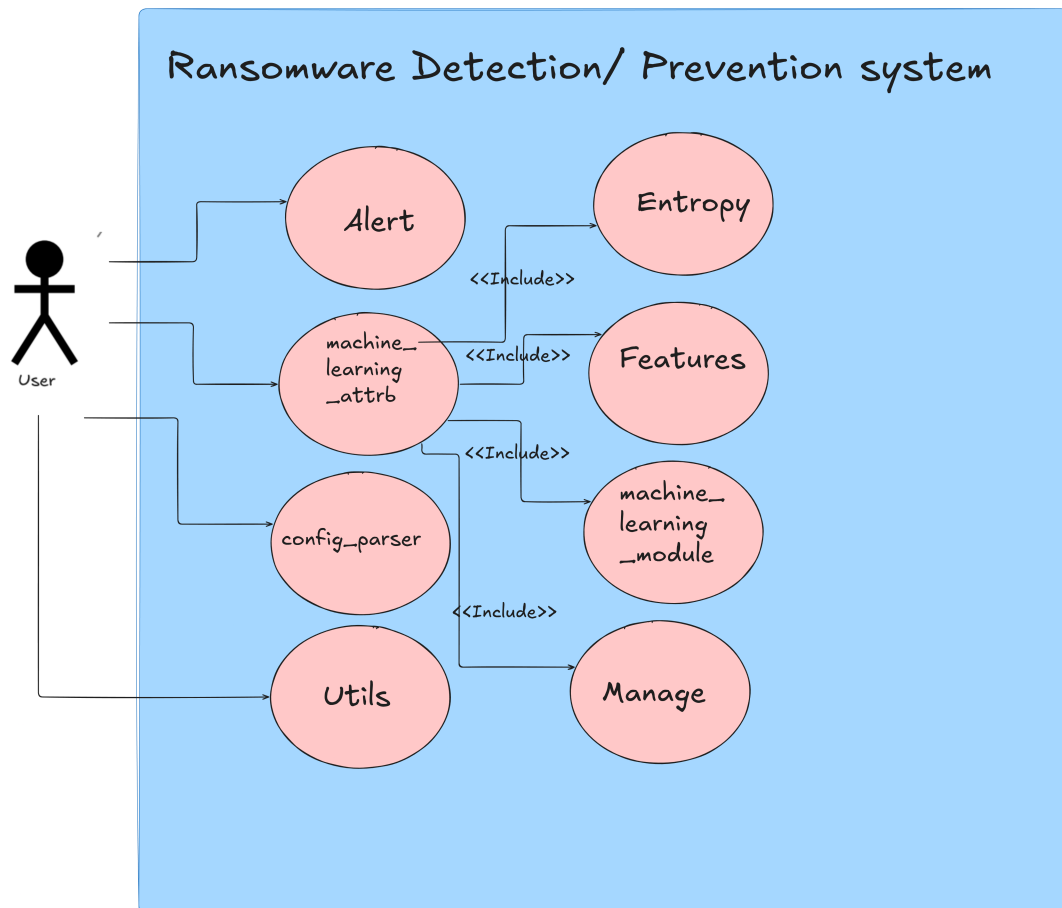


Figure 2: Use Case Diagram

4.4. Activity Diagram

This section go through the two main activity the program perform which include **Static Analysis** as well as **Dynamic Analysis**.

4.4.1. Static Analysis

Will start with the simplest of which. Static analysis is straight forward operation and demonstrate as follow. Figure 3

- The **main** function will start the thread reasonable for doing all the work regarding static analysis.
- All processes will be gathered constructing a **hash map** containing a key value pair of **PID & Path to executable**.
- a function will be invoke on the **hash map** to generate the **SHA256** hash and compare it with (Virusotal, n.d.) database.
- if match is found then a **Critical** severity alert will be sent. Other wise it will be discarded.

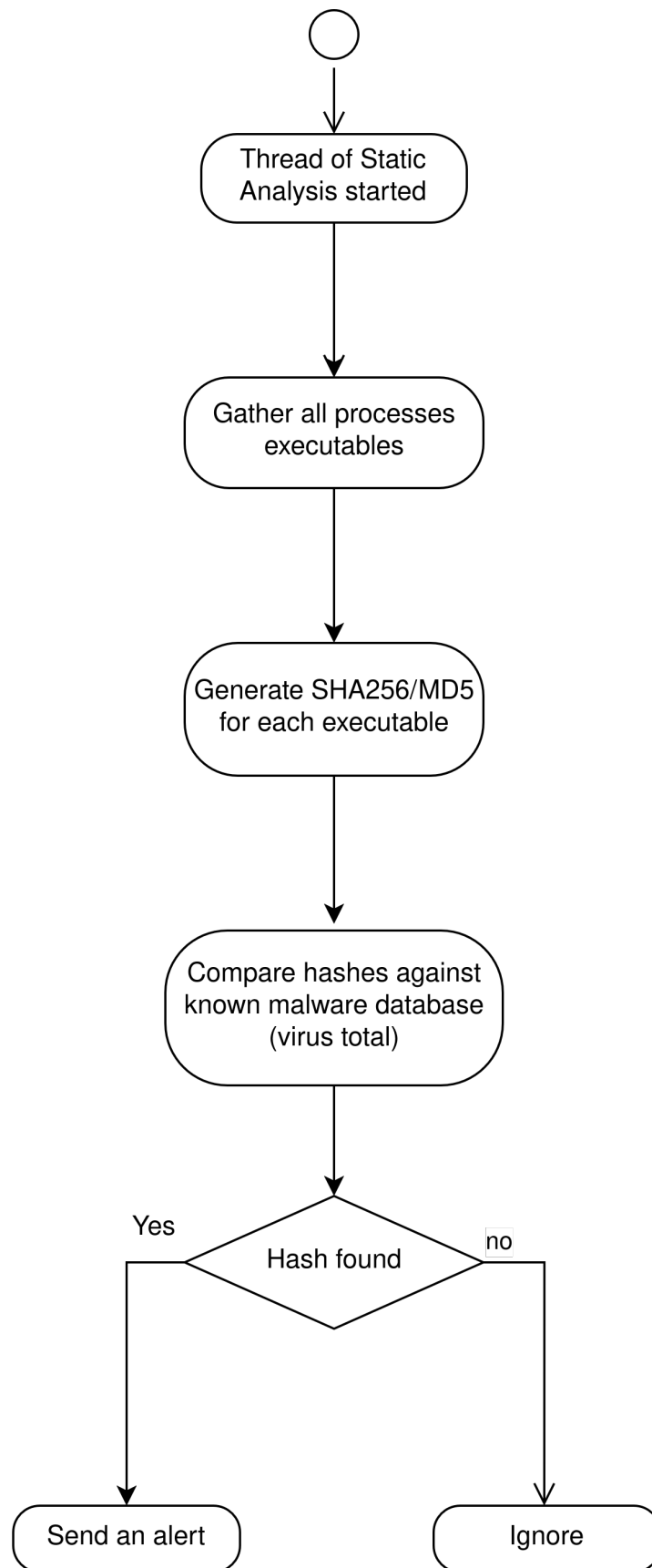


Figure 3: Static Analysis Activity Diagram

4.4.2. Dynamic Analysis

This section details the dynamic analysis process where all the complication starts.

Figure 4

1. First the program going to load all the modules and program found in the **Ebpf** file which is only a hooks that when a system call such read is triggered this function will be executing.
2. After loading the program and attaching it to the appropriate places. We going to load the **Data Structures** found in the **Ebpf** file. These a structures enable sharing data between a **kernel** module and a user program.
3. The most important of which is called **ProcMap** which contains a **Hash Map** store a pair of **PID** and a structure contains a counters for the number of system calls been made by the process.
4. Using this we are able to construct a data into our user program which we can **scale** and give it to the module to decide.
5. Another important **Map** is the **Buffer Map** which contains a **Hash Map** as well holds a **PID** and a buffer of size **256** that holds the written data generated by **write_syscall** in **LINUX**. Utilizing that go gather a sample **Entropy** for each process.
6. By now we have everything ready we hand the data to the machine learning module and as in the **Static analysis** it would alert if the prediction is higher than the **minimum severity to alert** other wise it would ignore it.

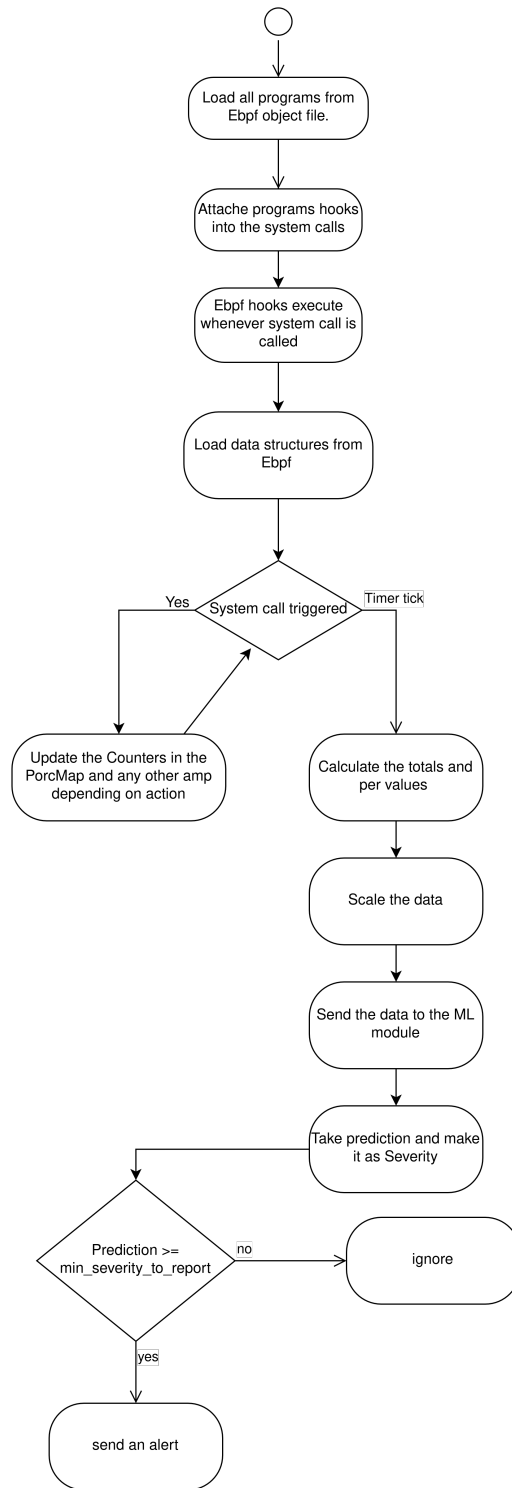


Figure 4: Static Analysis Activity Diagram

4.5. ML Training & Data Collection

This chapter covers the module implementation and datasets that have been used in the module. Basically (Hirano & Kobayashi, 2025), (Willers & Philippart, 2024) where that main two researches the dataset been used from. Let's cover them one by one explaining the data used from each

4.5.1. Ebpfangle Project.

The first paper and the main one (Willers & Philippart, 2024) covers relatively the same idea though as the author mention the dataset that been used to test the module were "Unbalanced" so an oversampling algorithm was applied to balance the dataset (Mohammed et al., 2020), helping and enabling us to balance the dataset to extend were it can be realistic and simulate real attacks. this data set consist of the following table Table 1

Table 1: Features existed in Ebpfangle dataset

Field	Description
PID	Process Identifier - Unique identifier for each process
Statistical Features	
C_max	Close Maximum - Maximum value of Close operations
C_sum	Close Sum - Total count of Close operations
D_max	Delete Maximum - Maximum value of Delete operations
D_sum	Delete Sum - Total count of Delete operations
O_max	Open Maximum - Maximum value of Open operations
O_sum	Open Sum - Total count of Open operations
P_max	Pattern Maximum - Maximum pattern value
P_sum	Pattern Sum - Total pattern count
Three-Operation Sequence Patterns	
CCC	Close-Close-Close - Sequential pattern of three Close operations
CCD	Close-Close-Delete - Pattern: Close → Close → Delete
CCO	Close-Close-Open - Pattern: Close → Close → Open
CDD	Close-Delete-Delete - Pattern: Close → Delete → Delete
CDO	Close-Delete-Open - Pattern: Close → Delete → Open

Field	Description
COC	Close-Open-Close - Pattern: Close $\rightarrow$ Open $\rightarrow$ Close
COD	Close-Open-Delete - Pattern: Close $\rightarrow$ Open $\rightarrow$ Delete
COO	Close-Open-Open - Pattern: Close $\rightarrow$ Open $\rightarrow$ Open
DCC	Delete-Close-Close - Pattern: Delete $\rightarrow$ Close $\rightarrow$ Close
DDD	Delete-Delete-Delete - Sequential pattern of three Delete operations
DDO	Delete-Delete-Open - Pattern: Delete $\rightarrow$ Delete $\rightarrow$ Open
DOD	Delete-Open-Delete - Pattern: Delete $\rightarrow$ Open $\rightarrow$ Delete
DOO	Delete-Open-Open - Pattern: Delete $\rightarrow$ Open $\rightarrow$ Open
OCC	Open-Close-Close - Pattern: Open $\rightarrow$ Close $\rightarrow$ Close
OCD	Open-Close-Delete - Pattern: Open $\rightarrow$ Close $\rightarrow$ Delete
OCO	Open-Close-Open - Pattern: Open $\rightarrow$ Close $\rightarrow$ Open
ODC	Open-Delete-Close - Pattern: Open $\rightarrow$ Delete $\rightarrow$ Close
ODD	Open-Delete-Delete - Pattern: Open $\rightarrow$ Delete $\rightarrow$ Delete
ODO	Open-Delete-Open - Pattern: Open $\rightarrow$ Delete $\rightarrow$ Open
OOC	Open-Open-Close - Pattern: Open $\rightarrow$ Open $\rightarrow$ Close
OOD	Open-Open-Delete - Pattern: Open $\rightarrow$ Open $\rightarrow$ Delete
OOO	Open-Open-Open - Sequential pattern of three Open operations

From the above dataset All features except PID were utilized as it does not indicate anything and the **PID** is given randomly from the system so it does not serve any business for me.

4.5.2. RanSMAP dataset.

The other paper (Hirano & Kobayashi, 2025) discuss and produce a new dataset for ransomware it capture events from RAM, Disk Access normal system calls. From this dataset, the Entropy values were utilized for the data written into disk which can be found in files that named **ata_write.csv** using this both datasets were successfully combined together and utilize it to train my module so that it able to detect using **Entropy** as well. The detailed data found in the **CSV** file can be found in the Table 2 below.

Table 2: Features existed in RANSMAP dataset

Field	Description
Temporal Information	
UNIX time in sec	UNIX Timestamp (Seconds) - The time of the I/O operation in seconds since the Unix epoch (January 1, 1970, 00:00:00 UTC). Provides second-level precision for when the disk operation occurred.
UNIX time in ns	UNIX Timestamp (Nanoseconds) - Additional nanosecond precision component of the timestamp. Combined with seconds, this provides high-resolution timing (up to nanosecond precision) for precise I/O operation sequencing and performance analysis.
Disk Location Information	
LBA	Logical Block Address - The logical block address on the storage device where the I/O operation is performed. LBA is used in modern storage systems to address blocks of data on disk, abstracting away physical disk geometry.
size	Operation Size - The size of the I/O operation in bytes (or blocks, depending on dataset specification). Indicates how much data was read from or written to the disk in this operation. Larger sizes may indicate bulk data transfers while smaller sizes might suggest metadata operations.
Data Characteristics	

Field	Description
entropy	Data Entropy - A measure of randomness or information density in the data being written or read, typically calculated using Shannon entropy. Values range from 0 (completely uniform data) to 8 (maximum randomness for byte-level entropy). High entropy may indicate encrypted or compressed data, which is relevant for detecting ransomware or encrypted malware payloads.

4.5.3. Final Dataset.

Basically we combine the first dataset (Willers & Philippart, 2024) and the **Entropy** from (Hirano & Kobayashi, 2025) dataset forming a dataset that can be used to train the module and be able to detect Ransomware attacks in its early stages. Table below shows the final dataset ignoring the **LABEL** flag though it exist in the actual dataset.

Table 3: Features will be used to train the module

Field	Description
Statistical Features	
C_max	Close Maximum - Maximum value of Close operations
C_sum	Close Sum - Total count of Close operations
D_max	Delete Maximum - Maximum value of Delete operations
D_sum	Delete Sum - Total count of Delete operations
O_max	Open Maximum - Maximum value of Open operations
O_sum	Open Sum - Total count of Open operations
P_max	Pattern Maximum - Maximum pattern value
P_sum	Pattern Sum - Total pattern count
Three-Operation Sequence Patterns	
CCC	Close-Close-Close - Sequential pattern of three Close operations
CCD	Close-Close-Delete - Pattern: Close → Close → Delete
CCO	Close-Close-Open - Pattern: Close → Close → Open
CDD	Close-Delete-Delete - Pattern: Close → Delete → Delete
CDO	Close-Delete-Open - Pattern: Close → Delete → Open
COC	Close-Open-Close - Pattern: Close → Open → Close
COD	Close-Open-Delete - Pattern: Close → Open → Delete
COO	Close-Open-Open - Pattern: Close → Open → Open
DCC	Delete-Close-Close - Pattern: Delete → Close → Close
DDD	Delete-Delete-Delete - Sequential pattern of three Delete operations
DDO	Delete-Delete-Open - Pattern: Delete → Delete → Open
DOD	Delete-Open-Delete - Pattern: Delete → Open → Delete

Field	Description
DOO	Delete-Open-Open - Pattern: Delete → Open → Open
OCC	Open-Close-Close - Pattern: Open → Close → Close
OCD	Open-Close-Delete - Pattern: Open → Close → Delete
OCO	Open-Close-Open - Pattern: Open → Close → Open
ODC	Open-Delete-Close - Pattern: Open → Delete → Close
ODD	Open-Delete-Delete - Pattern: Open → Delete → Delete
ODO	Open-Delete-Open - Pattern: Open → Delete → Open
OOC	Open-Open-Close - Pattern: Open → Open → Close
OOD	Open-Open-Delete - Pattern: Open → Open → Delete
OOO	Open-Open-Open - Sequential pattern of three Open operations
Data Characteristics	
entropy	Data Entropy - A measure of randomness or information density in the data being accessed, calculated using Shannon entropy. Values range from 0 (completely uniform data) to 8 (maximum randomness for byte-level entropy). High entropy indicates encrypted or compressed data, which is a key indicator for detecting ransomware and encrypted malware payloads. When combined with file operation patterns, entropy provides crucial insight into whether malicious processes are encrypting files.

4.5.4. Random Forest.

Also as discussed in Section 2 specially at Section 2.7 section the pros and cons of using **Random Forest** algorithm to train the module over all it consider one of the best algorithm in machine learning generally therefore, it was selected for this implementation.

4.5.5. ML Model Performance Evaluation

After taring the module we can observe the following metrics found in the table below Table 4

Table 4: module performance

Metric	Value	Definition & Interpretation
Accuracy	0.9758	The proportion of correct predictions (both true positives and true negatives) among all predictions. It answers: "What percentage of all predictions were correct?" Formula: $(TP + TN) / (TP + TN + FP + FN)$. High accuracy (97.58%) indicates the model correctly classifies most instances, though it can be misleading with imbalanced datasets.
Precision	0.9565	The proportion of true positive predictions among all positive predictions made by the model. It answers: "When the model predicts positive, how often is it correct?" Formula: $TP / (TP + FP)$. High precision (95.65%) means that when the model identifies something as malware, it is correct 95.65% of the time, with few false alarms.
Recall	0.8800	The proportion of actual positive cases that were correctly identified by the model. Also called Sensitivity or True Positive Rate. It answers: "Of all actual positive cases, how many did the model find?" Formula: $TP / (TP + FN)$. Recall of 88% means the model successfully detects 88% of actual malware instances, but misses 12%.
F1-score	0.9167	F1 Score is a performance metric used in machine learning to evaluate how well a classification model performs on a dataset especially when the classes are imbalanced meaning one class appears much more frequently than another. It is the harmonic mean of precision and recall which combine both metrics into a single value that balances their importance

Metric	Value	Definition & Interpretation
ROC-AUC	0.9034	AUC-ROC curve is a graph used to check how well a binary classification model works. It helps us to understand how well the model separates the positive cases like people with a disease from the negative cases like people without the disease at different threshold level

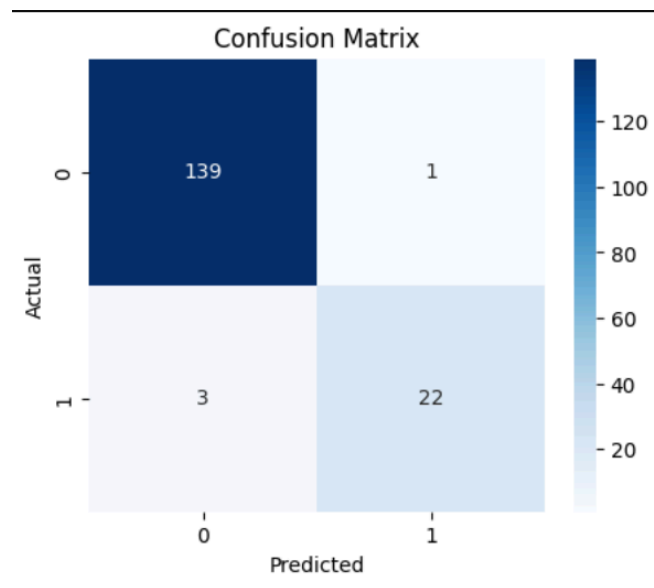


Figure 5: Confusion Matrix

The confusion matrix is a sample diagram that shows how well the module has classified each test that it been test from the image above we can see that the module has correctly classified 161 test has been correctly classified while 4 of the remaining test cases where incorrectly classified this consider a very good metric for the current module when the current limited available datasets.

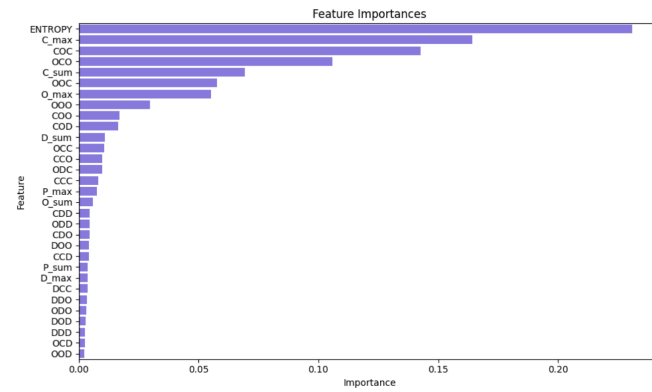


Figure 6: Importance metrics

The **Importance Metric** chart is very simple chart it only hilight the features and arrange them according to their importance when evaluating a vector. As shown in the figure above **Entropy** is consider the most important feature along with **C_max** as second most important also patterns such **OOD** and **OCD** are the least important ones.

4.6. Extended Berkeley Packet Filter (eBPF)

eBPF is a technology that is very great it enable developers to run code in the Linux kernel level. Basically this code is runs in a virtual machine similar on how **Java** works. They very simple and remove the overhead that comes when developing on the pure kernel. Specially in their ways to enable sharing data from **kernel space** to **user space**.

The main idea of **eBPF** that is generate custom bytes when compiled and run it in the kernel while maintaining safely using the verifier which is also similar to the **Rust compiler**

4.6.1. eBPF Programs:

eBPF programs are typically functions that labeled with special label called SEC which take a prameter refers to the system call we want to hook and for our function to run whenever that system call is called.

4.6.2. eBPF Verifier:

Verifier in simple terms like your parent will teach you how to do things the right way but you will suffer. It very had to satisfy then it helps in keeping everything running on the kernel secure and error free.

4.6.3. eBPF Maps:

These are efficient key-value data structures that enable communication between **eBPF** programs running in kernel space and user-space applications. In the system, we utilize a **Hash Map** where the key is the **PID** and the value contains aggregated metrics about process behavior including system call counts and operation patterns. The maps provide a high-performance mechanism for sharing collected data with our **Rust** client program for analysis.

4.6.4. eBPF Maps:

Maps in **eBPF** is the most amazing feature existing in **eBPF** still studying it on how sophisticated it is its a form of data storage you can have Ring buffers, HashMaps, Arrays, etc... these maps are easy to use and deal with they are fast and secure they can be called from the **user space** code to retrieve data from the **kernel space code**.

4.6.5. Hook Points:

eBPF programs attach to certain events type there are plenty of them but the most common used are **Tracepoints** and **Kprobes** they kinda similar on how they serve though **kprobes** offers a better interface to handle data and less control over the event data. While on the other hand **Tracepoints** is the complete opposite

The beauty of **eBPF** lies in its ability to provide kernel-level observability with near-zero overhead while maintaining system safety and stability. This makes it the ideal foundation for our real-time malware detection system.

4.7. Class Diagram

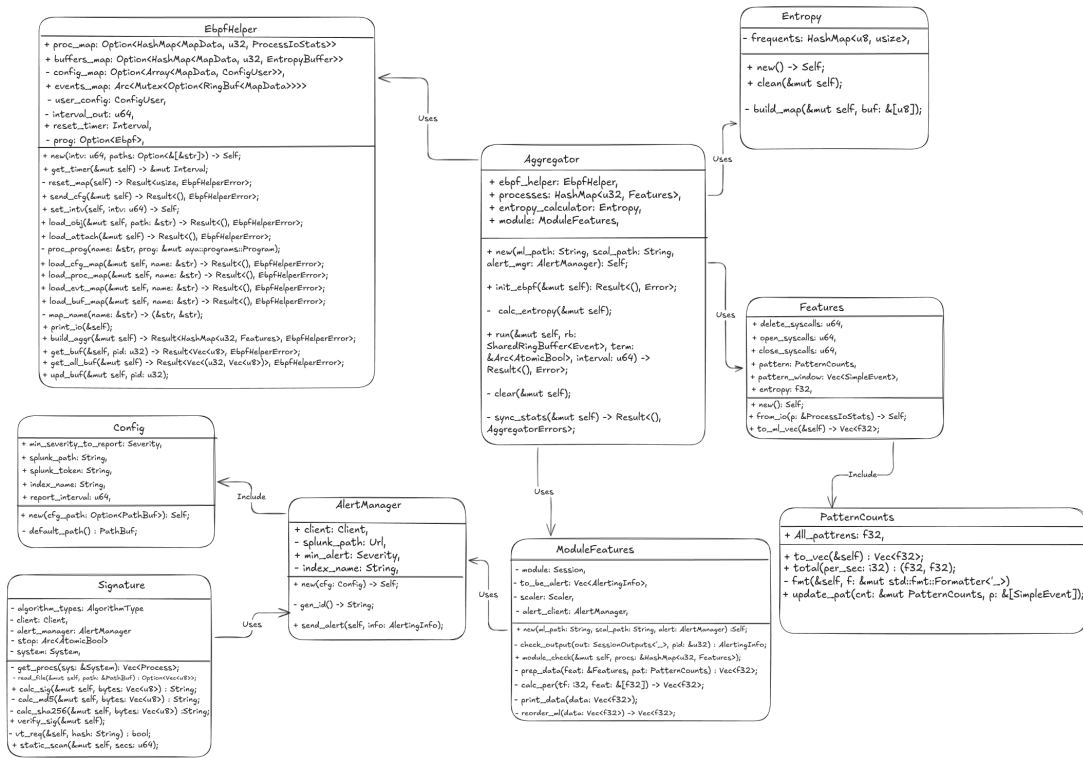


Figure 7: Class Diagram

4.7.1. Overview

Basically the figure Figure 7 shows the important class that been used in the project not all classes were included for clarity, nevertheless these shows the basic component and the most important that used to build the application. You will notice that every class has a **new** function, you can relate this to a constructor of an object. Using the **new** function is the **RUST** way of doing object work.

4.7.2. Aggregator

As shown in the diagram, the **Aggregator** class is the bare-bones implementation. It connects the other classes with each other. **Aggregator** class is design or the **Dynamic Analysis** side of things.

The **run** function found in there is the main loop of the program execution when the timer tick it will gather all the require data for the **ML** to decide.

4.7.3. EbpfHelper

This is the second most important class in the project as it responsible for all the require work for **Ebpf**.

- The loader functions basically load either program or map from the **Ebpf** program.
- The get functions help us get the information from **Ebpf** maps whenever we need them but without them it totally fine we can access the map and it will be sync with whatever exist on the map from the kernel side. Nevertheless having get function save me from the headache involve with **Rust ownership module**.
- Resetting function will just reset everything in the maps to avoid overwhelming the memory.

4.7.4. Entropy

This class is just as its name implies just a helper to calculate the **entropy**

4.7.5. Config

This class is used to parse user configuration found in the **root** directory. If it does not find a path or the file there then it will use a default configuration but surly it does not contains the **splunk_token**. There is no function in the class that indicate the config loading, But the new function does that here.

4.7.6. Signature

This class is used to do perform signature checking for the files. It also suppose to have an api key for (Virusotal, n.d.). It also support two type of hashing.

- MD5.
- SHA256.

4.7.7. AlertManager

This class is responsible for sending alerts. The client is and **HTTP** client used to send these notifications.

4.7.8. ModuleFeature

This class is one of the important ones. It does all the **ML** work.

- Prepare function will make sure that the data is scaled.
- the order function will make sure that the order of the feature is correct to be used in the **ML**.

4.7.9. Features & PatternCount

These two classes represent the features them self.

- **Features**: This class is used for the main features including the **Entropy**.
- **Pattern count**: This classes is used for the **patterns** discussed earlier.

4.8. Sequence Diagram

4.8.1. Overview

This section illustrates the activities required to complete both tasks **Static** as well as **Dynamic** analysis.

4.8.2. Static Analysis

The Static Analysis section is covered first. The process as was mention earlier is quite simple when it comes to **Static Analysis**. The required steps are as follows:

1. Start the thread execution of **Static Analysis**
2. The next step involves gathering all the active processes along with their executable if exist.
3. A hash must then be generated either **MD5** or **SHA256** hash to be able to compare with (Virusotal, n.d.) database.
4. Compare with (Virusotal, n.d.) database
5. If match found then alert will be sent with **Critical** severity.
6. Other wise the operation will be discarded.

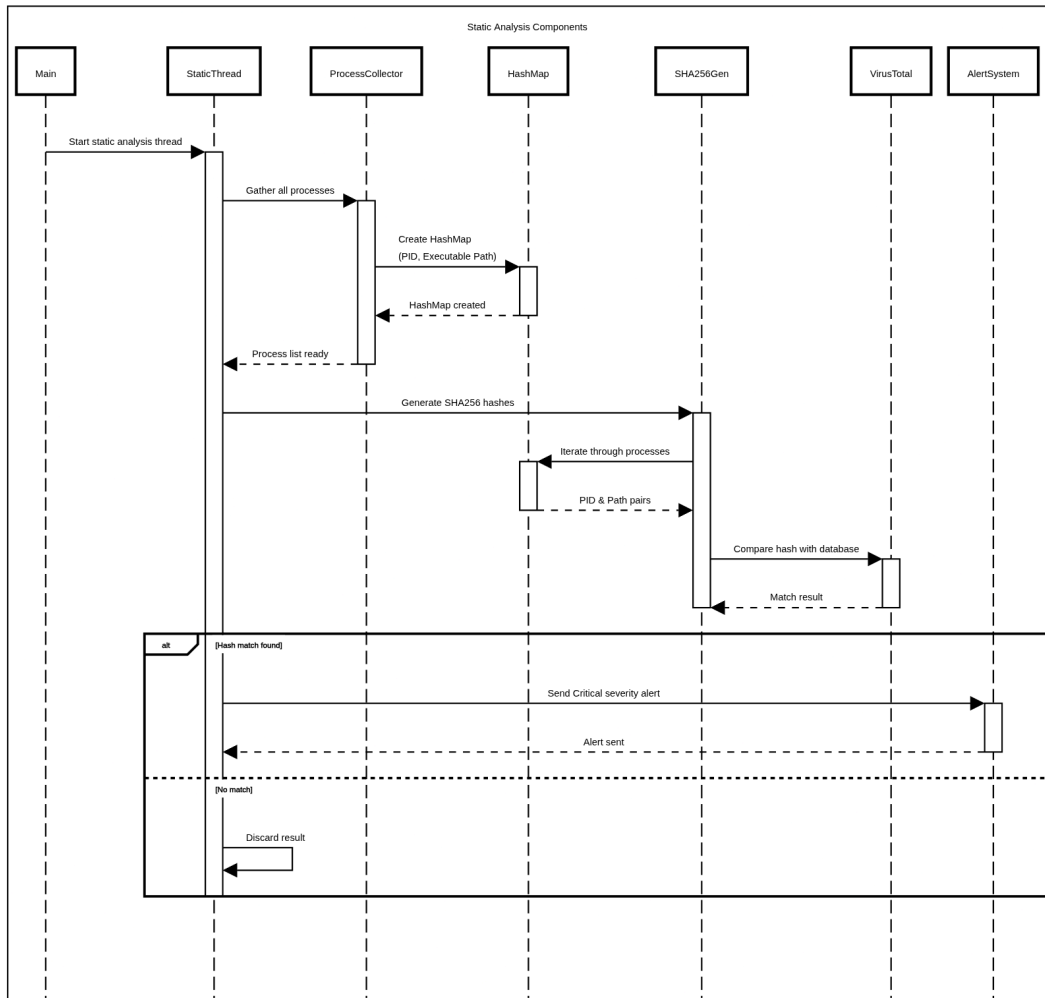


Figure 8: Static Analysis Sequence Diagram

4.8.3. Dynamic Analysis

The Dynamic Analysis section follows, the process as mentioned earlier is where all the complication starts when it comes to Dynamic Analysis. The required steps are as follows:

1. First the program going to load all the modules and program found in the Ebpf file which is only a hooks that when a system call such read is triggered this function will be executing.
2. After loading the program the programs must be attached to the appropriate places in the kernel.
3. The Data Structures must then be loaded which can be found in the Ebpf file. These Data structures enable sharing data between a kernel module and a user program.
4. The most important of which is called ProcMap which contains a Hash Map store a pair of PID and a structure contains a counters for the number of system calls been made by the process.
5. Using this data can be constructed in the user space which then can be scale and give it to the module to decide.
6. Another important Map is the Buffer Map which contains a Hash Map as well holds a PID and a buffer of size 256 that holds the written data generated by write_syscall in LINUX. Utilizing that to gather a sample Entropy for each process.
7. Once data preparation is complete, it is passed to the machine learning module.
8. If the prediction is higher than the minimum severity to alert then alert will be sent.
9. Other wise it would be ignored.

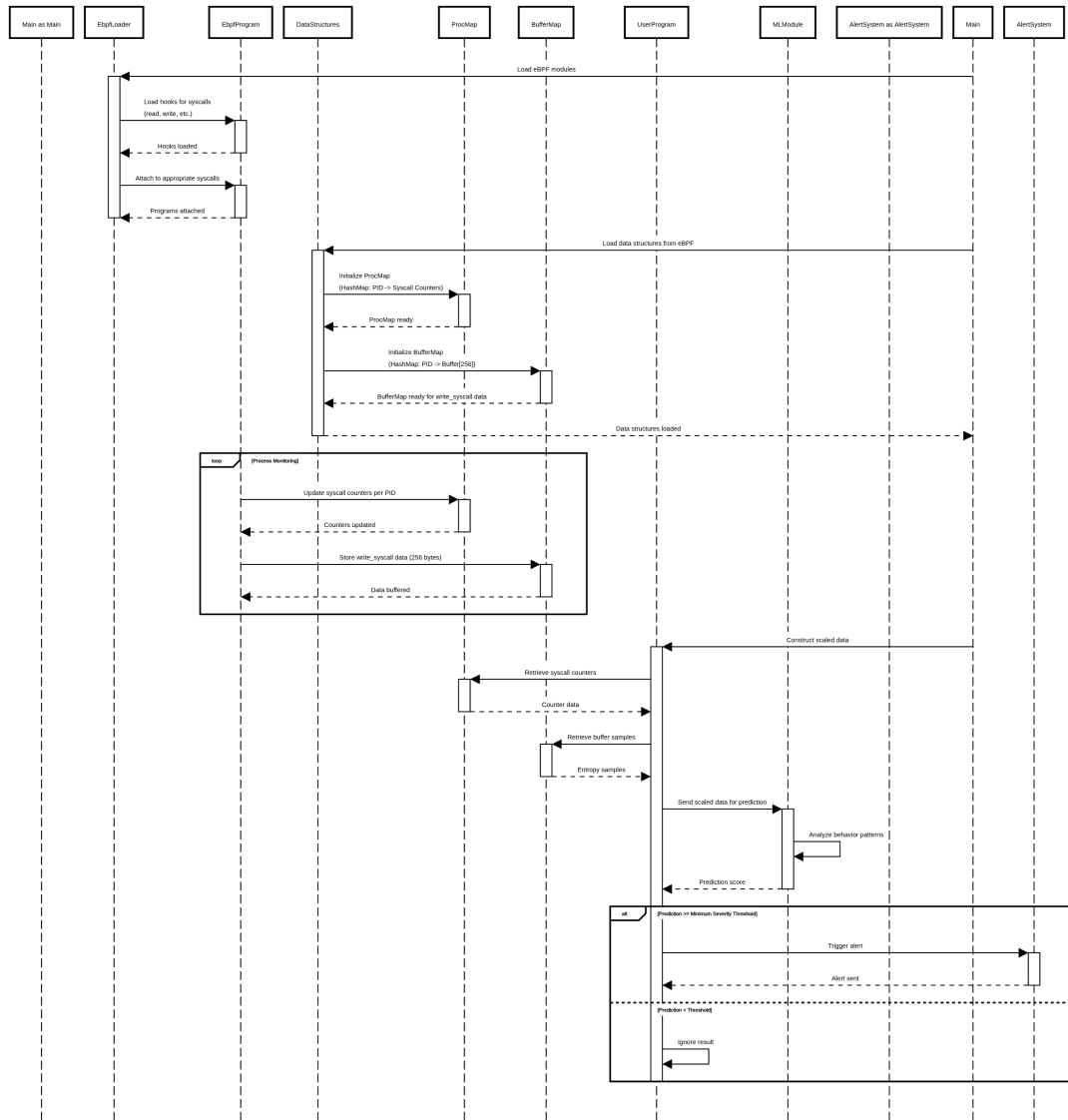


Figure 9: Dynamic Analysis Sequence Diagram

4.9. Software Architecture Diagram

Software architecture diagram Figure 10 shows the internal component in the system and what are the system boundaries.

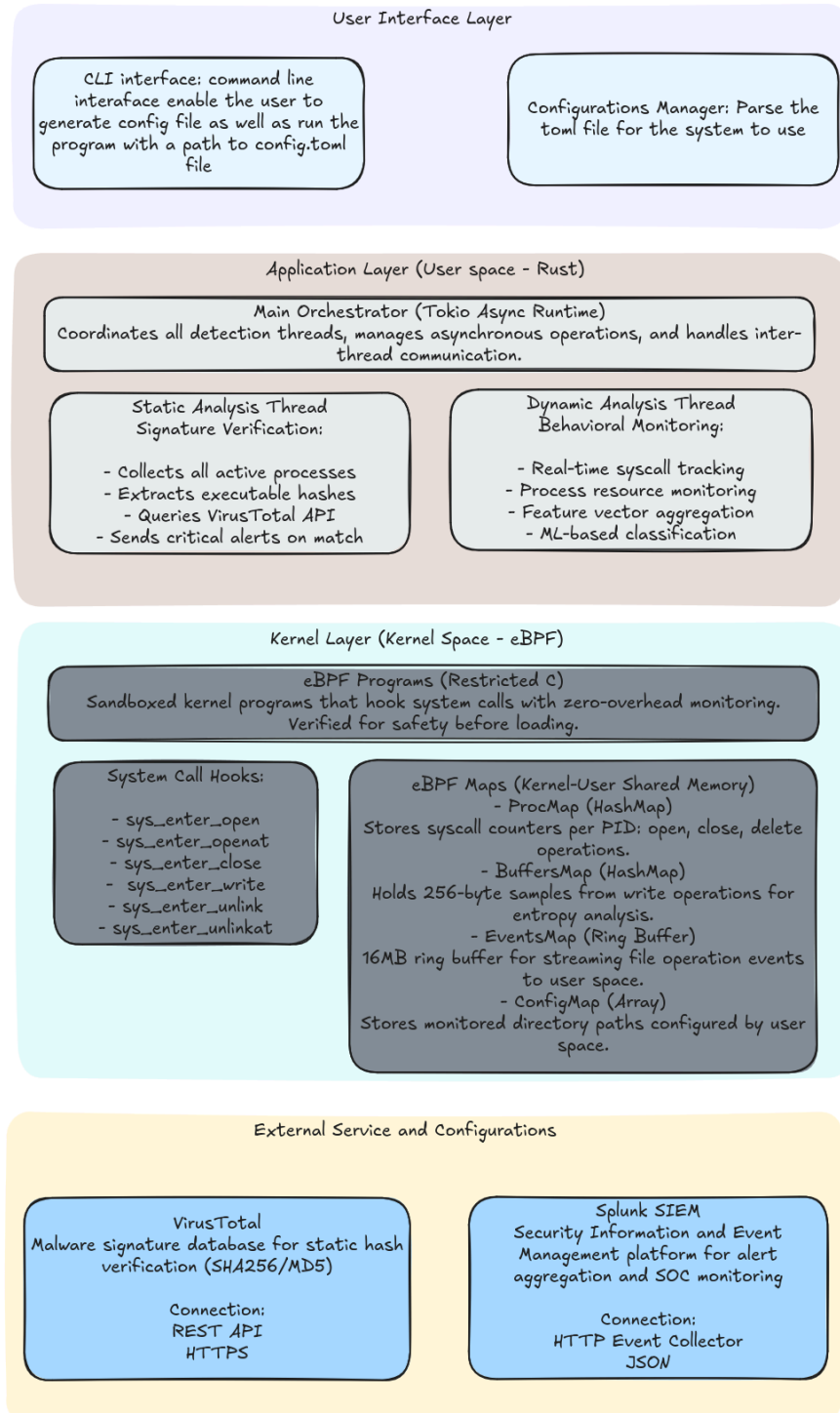


Figure 10: Software Architecture Diagram

4.9.1. User interface

User interface here refers to components in which the user has direct interaction with. CLI interface which enable the user to generate a configuration file or run the system with a path to configuration file. The code snippet below shows the CLI interface.

```
Usage: crab_shield <COMMAND>

Commands:
  run                Run CrabShield with the specified configuration
  generate-config    Generate a default configuration file
  help              Print this message or the help of the given
subcommand(s)

Options:
  -h, --help        Print help (see more with '--help')
  -V, --version      Print version
```

Each of these commands has a sub flag called verbose which enable the user to see all the internal operation happening in a beautiful logging format.

```
[2026-01-28T05:22:40Z INFO crab_shield::config_parser::parser]
Config file has been read from "/root/.crab_shield.toml"
[2026-01-28T05:22:40Z INFO crab_shield::config_parser::parser]
Config file has been parsed successfully.
[2026-01-28T05:22:40Z INFO crab_shield] Starting system...
[2026-01-28T05:22:40Z INFO
crab_shield::machine_learning_module::machine_learning_module]
Module configuration has been loaded
[2026-01-28T05:22:40Z INFO crab_shield] Done init aggregator
[2026-01-28T05:22:40Z INFO crab_shield] Done Starting Static
Analysis
[2026-01-28T05:22:40Z INFO crab_shield::manage::ebpf_helper]
Object file opened successfully
[2026-01-28T05:22:40Z INFO crab_shield::manage::ebpf_helper] Array
map ConfigMap loaded
```

Run option has a flag which is optional allows the user to specie a path to a configuration file by default it will assume the default configuration file at /root/.crab_shield.

4.10. Deployment Diagram

Finally Deployment diagram Figure 11 explain the over all image on the system.

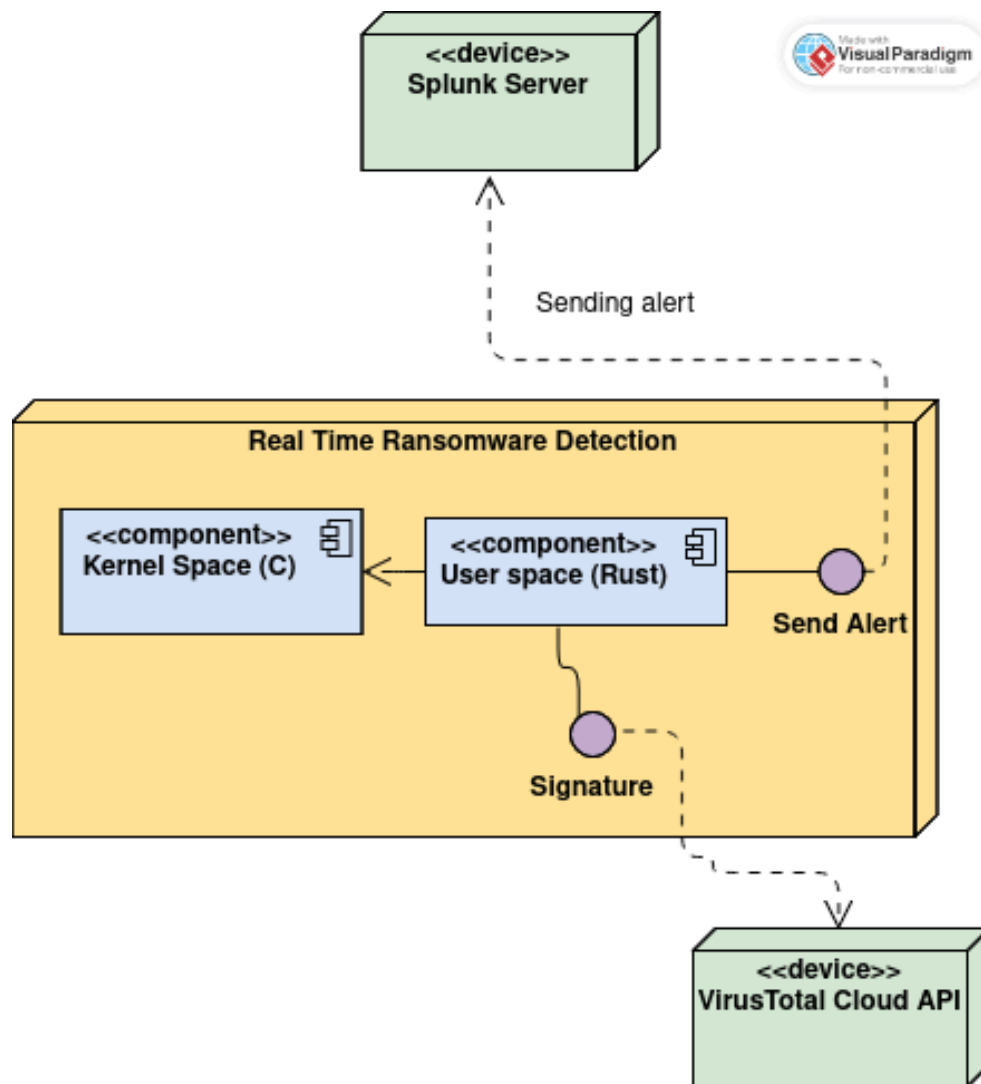


Figure 11: Deployment Diagram

As shown in the diagram above, the main system is running say on a server or an endpoint device. It will be running the Rust user space code as well as the C kernel space code. The Virustotal is an external API which we need to access from the internet. As for Splunk it can be running on the same local machine. However an assumption here has been made that in a normal corporate environment Splunk usually run on an external server that gather the events from every other system.

5. Implementation

5.1. Overview

This chapter discusses the actual implementation of the system, covering the development environment, tools used, and the step-by-step process of building each component. This includes explanations of how the design discussed in Chapter 4 was translated into working code, the challenges encountered during implementation, and the solutions developed to overcome them.

5.2. Development Environment and Tools

5.2.1. Operating System and Kernel

The project has been developed on the following **Linux** distribution and kernel version.

- **Distribution:** Arch linux (Arch Linux Project, 2026).
- **kernel version:** Linux 6.17.9-arch1-1

The project requires a Linux system with kernel support for **eBPF** features including:

- eBPF ring buffers for efficient event streaming
- eBPF hash maps for data sharing between kernel and user space
- BTF (BPF Type Format) support for CO-RE (Compile Once - Run Everywhere)
- Support for tracepoints on system calls

5.2.2. Programming Languages and Versions

The implementation uses two main programming languages:

- **Rust** - Used for the main application including eBPF loading, data processing, and system interactions with the kernel. The version of **Rust** that has been used to develop the project is `rustc 1.92.0`.
- **C** - Used for writing eBPF programs that run in kernel space. And compiled using `clang` compiler as `gcc` does not support compiling a **eBPF**.
- **Python 3.13+** - Used for machine learning model training and dataset preparation.

5.2.3. Rust Dependencies and Libraries

The project relies on several Rust crates to handle different aspects of the implementation:

Core eBPF Libraries:

- aya (custom branch from altugbozkurt07/aya as the main lib is yet to implement using **kernel functions**) - The main eBPF library for Rust, providing APIs to load, attach, and manage eBPF programs. A custom branch was utilized that includes support for ksym (kernel symbols) helpers needed for string operations in kernel space.
- aya-log (0.2.1) - Enables logging from eBPF programs to user space for debugging purposes.

Data Processing:

- ndarray (0.17.1) - Used for array operations when preparing data for machine learning inference.

Machine Learning:

- ort (2.0.0-rc.11) - ONNX (Kim et al., 2022) Runtime for Rust, allows loading and running the trained Random Forest model in production.

Networking and API:

- reqwest (0.13.1) - HTTP client for sending alerts to Splunk and querying VirusTotal API.
- tokio (1.48.0) - Async runtime for handling concurrent operations.

Serialization:

- serde (1.0.228) - Serialization framework for Rust.
- serde_json (1.0.149) - JSON support for API communication.
- toml (0.9.11) and serde_toml (0.0.1) - Configuration file parsing.

Utilities:

- anyhow (1.0.100) - Error handling with context.
- log (0.4.29) and env_logger (0.11.8) - Logging infrastructure.
- uuid (1.19.0) - Unique identifier generation for alerts.

5.2.4. Python Dependencies for Machine Learning

The machine learning training pipeline uses the following Python libraries:

- `scikit-learn` (1.8.0) - Machine learning framework, used for Random Forest implementation.
- `pandas` (2.3.3) - Allows to handle data very effectively.
- `numpy` (2.4.0) - Numerical computing.
- `imblearn` - Provide an interface for Oversampling algorithm
- `matplotlib` (3.10.8) and `seaborn` (0.13.2) - Allow making an images to visualise performance
- `skl2onnx` (1.19.1) - Converts scikit-learn models to ONNX format for deployment.
- `onnxruntime` (1.23.2) - Help in converting from skLearn to ONNX format.
- `jupyterlab` (4.5.1) - An interactive feature on top of python helping in making the code easier to read and faster to interpret.

5.2.5. Development Tools

The main tools used in this project are as follows:

- **IDE: Neovim** (Neovim Project, 2026).
- **Version Control:** Git/ GitHub (Git Community, 2026).
- **Documentations: Typst** (Typst Project, 2026).
- **Cargo:** for **Rust** package management.

5.3. eBPF Program Implementation

As discussed earlier eBPF is the main part of the code that allow the program to access real time access to system calls via hooking.

5.3.1. eBPF Program Structure

The **eBPF** programs are usually written in C nevertheless Rust also provide a way via aya to write **eBPF** programs in Rust. The code snippets below shows the most important part of the code.

Header Files and Constants:

```
#define MAX_CHUNK_SIZE 64
#define MAX_CHUNK_SIZE_MID 128
#define MAX_CHUNK_SIZE_BIG 256
#define MAX_ENTRIES 102400
#define SAMPLE_DATA_SIZE 256
#define MAX_PATH_NUMBER 5
#define MAX_PATH_LENGTH 126
```

These constant values are mostly to satisfy the **Verifier** though they help of making the code more readable. *Note:* These constants are subject to change and can be changed.

5.3.2. Data Structures

Using what discussed earlier **eBPF maps** to store the needed data.

ProcInfo Structure: This structure tracks system call counts for each process:

```
struct ProcInfo {
    __u64 open_syscall;
    __u64 close_syscall;
    __u64 delete_syscall;
};
```

EntropyBuf Structure: This structure helps in providing buffers for each process when it calls the **write** system call to evaluate the Entropy.

```
struct EntropyBuf {  
    char buf[SAMPLE_DATA_SIZE];  
    bool is_handled;  
    __u64 size_of_written_data;  
    bool ready_to_be_handled;  
};
```

The flags `is_handled` and `ready_to_be_handled` coordinate between kernel and user space to ensure data consistency.

Event Structure: Used for streaming events through the ring buffer:

```
typedef enum EventType {  
    EVENT_OPEN = 1,  
    EVENT_CLOSE = 3,  
    EVENT_DELETE = 5,  
} EventType;  
  
struct Event {  
    __u32 pid;  
    EventType event_type;  
};
```

5.3.3. eBPF Maps

The program uses several map types to store and share data:

ProcMap (Hash Map): Stores ProcInfo structures indexed by PID, tracking system call counts for each monitored process.

BuffersMap (Hash Map): Stores EntropyBuf structures indexed by PID, holding sample data from write operations.

EventsMap (Ring Buffer): A 16MB ring buffer for streaming events from kernel to user space. This allows real-time monitoring of file operations without blocking.

ConfigMap (Array): Stores configuration data including the paths to monitor. This allows user space to configure the eBPF program's behavior.

5.3.4. System Call Hooks

The eBPF program attaches to several system call tracepoints:

Open Operations (sys_enter_open and sys_enter_openat): These hooks monitor file open operations. When a process opens a file in a protected path, the hook:

1. Extracts the PID and file path from the system call arguments
2. Checks if the path matches any configured protected paths using `bpf_loop` and `bpf_strstr`
3. If matched then increase the open counter and inserts an event into the ring buffer with letter O for open.

Close Operations (sys_enter_close): If a close system call is called then this function will perform as explain above expect now it will going to insert C for close.

Delete Operations (sys_enter_unlink and sys_enter_unlinkat): This two functions will watch for these two system calls which associate with deleting files if they called same thing will happen as above.

Write Operations (sys_enter_write): This is one of the most complex hooks. It captures sample data from write operations to calculate entropy:

1. Checks if the process is already being monitored
2. If the buffer is ready for new data, reads up to MAX_CHUNK_SIZE_BIG (256 bytes) from the write buffer
3. Marks the buffer as ready for user space to process

5.3.5. Path Matching Implementation

One of the key features is monitoring only specific paths. The implementation uses `bpf_loop` to iterate through configured paths:

```
static long is_path_protected(__u64 index, void* ctx) {
    struct loop_ctx* loop_ctx = ctx;
    const struct ConfigUser* config = get_config();

    if (index >= MAX_PATH_NUMBER) return 0;

    const char* path_config = config->paths[index];
    if (!path_config) return 0;

    if (bpf_strstr(loop_ctx->path, path_config) != -ENOENT) {
        return 1; // Path matches
    }
    return 0;
}
```

This approach allows flexible configuration of monitored directories without recompiling the eBPF program.

5.3.6. eBPF Verifier Challenges

The eBPF verifier is quite strict about program safety. The verifier exhibits unusual behavior in that if you look closely at the code you can find that there are 3 constants that represent chunks which is kinda odd but that's only to satisfy the **verifier** as it does not allow for a user space variable to be used in kernel operations while the security implications are understood of that behavior, nevertheless if proper checks were in place there is no reason to restrict that kind of behavior.

5.3.7. Compilation Process

To compile the **eBPF** source code As mentioned earlier, Clang was used the following command will be used to compile it.

```
clang -O2 -mllvm -D__TARGET_ARCH_x86 -g -target bpf -c  
"$SOURCE_BPF" -o "$OBJECT_FILE"
```

- **O2**: meaning level 2 optimizations.
- **-mllvm**: this flag means the following flags for **llvm** not for **clang** it self.
- **D-D__TARGET_ARCH_x86** : define a preprocessor macro
- **-g** : enable debugging flags.
- **-target bpf**: this tells **clang** to generate bytecode for **BPF**.
- **-c**: for the source file.
- **-o**: for the output file name.

5.4. Rust Application Implementation

The Rust application serves as the user space component, loading eBPF programs, processing collected data, and making detection decisions.

5.4.1. Project Structure and Main Components

The application is organized into several key classes as shown in the class diagram in Chapter 4:

- **Aggregator** - Orchestrates the dynamic analysis process
- **EbpfHelper** - Manages eBPF program loading and map access
- **Entropy** - Calculates Shannon entropy from sample data
- **Config** - Parses and manages configuration
- **Signature** - Handles static analysis and hash comparison
- **AlertManager** - Sends alerts to Splunk
- **ModuleFeature** - Prepares data for ML inference
- **Features & PatternCount** - Represent the feature vectors

5.4.2. EbpfHelper Implementation

The EbpfHelper class is responsible for all **eBPF** operation it provide an interface to perform all activities with **eBPF**

Loading eBPF Programs: This module provide a loading function which basiclly runs once the program start executing they will help in loading all the require programs and maps to **user space**.

1. `load_object_file` this function will load the object file it self.
2. `load_modules_and_attach` this function will make sure that all programs are loaded and attached into the correct hooks.

Accessing Maps: The get functions provide safe access to eBPF maps:

- `construct_aggregator` this function will collect the current existing **Aggregator** features from the kernel space and pass them to the user space function.
- `get_all_buffers` this function will access the map shown earlier `BuffersMap` to collect all currently existing buffers and pass them to the user space.
- `send_config` this function responsible to send configurations found in the configuration file to the kernel space.

Resetting Maps: Resetting process happen when the data is being access and already stored in the user space program therefor we do not need it be kept in the kernel space so we just clear it on access.

5.4.3. Aggregator Implementation

The Aggregator is the heart of the dynamic analysis loop. The `run()` function implements the main detection cycle:

1. The function will start by loading some timers. Helping in reporting process and accessing the **Ring Buffer (EventsMap)** to process events.
2. If **report_interval** ticks it will do the following.
3. Synchronizes all the processes from the kernel space structures to the user space.
4. Then **Entropy** will be calculated.
5. After that everything will be passed to the module to check.
6. Finally **clear** function will be called to clear all data that has been consumed and not needed.

```

pub async fn run(
    &mut self,
    ring_buffer: SharedRingBuffer<Event>,
    terminator: &Arc<AtomicBool>,
    report_interval: u64,
) -> Result<(), AggregatorErrors> {
    info!("Starting main loop");
    let mut tick =
tokio::time::interval(Duration::from_millis(500));
    let mut reset =
tokio::time::interval(Duration::from_secs(report_interval));

    while terminator.load(Ordering::Acquire) {
        tokio::select! {
            _ = tick.tick() => {
                // 1. Consume kernel events
                process_events(ring_buffer.clone(), &mut
self.processes);
            }
            _ = reset.tick() => {

                self.sync_proc_stats().await?;
                self.calculate_entropy_for_processes().await;

                self.module.module_check(&self.processes).await;
                self.clear().await;

            }
        }

        Ok(())
    }
}

```

5.4.4. Entropy Calculation

The Entropy class implements Shannon entropy calculation:

$$H(X) = - \sum P(x) \cdot \log_2(P(x))$$

Where P(x) is the probability of each byte value appearing in the sample. The implementation:

1. Counts frequency of each byte value (0-255)
2. Calculates probability for each byte
3. Applies Shannon formula
4. Returns entropy value between 0 (uniform data) and 8 (maximum randomness)

```
pub async fn calculate_entropy(&mut self, buf: Vec<u8>) -> f32
{
    self.clean();
    self.construct_hash_map(&buf);
    let mut entropy = 0.0;
    for freq in self.frequents.values() {
        let p = *freq as f64 / buf.len() as f64;
        if p > 0.0 {
            entropy += -p * p.log2();
        }
    }

    // Return entropy but not tested yet
    info!("Finish calculating entropy for a size of {}",
buf.len());
    entropy as f32
}

fn construct_hash_map(&mut self, buf: &[u8]) {
    if buf.is_empty() {
        println!("Make sure to read first");
        return;
    }
    for byte in buf.iter() {
        self.frequents
```

```
        .entry(*byte)
        .and_modify(|count| *count += 1)
        .or_insert(1);
    }
}
```

High entropy values (above 7.5) typically indicate encrypted or compressed data, a strong indicator of ransomware activity.

5.4.5. Configuration Management

The Config class handles user configuration from a TOML file. The configuration includes:

```
pub struct Config {
    // Minimum severity level required for an event to be reported
    pub min_severity_to_report: Severity,

    // HTTP endpoint or URL path used to send data to Splunk
    pub splunk_path: String,

    // Authentication token used to authorize requests to Splunk
    pub splunk_token: String,

    // Name of the Splunk index where logs/events will be stored
    pub index_name: String,

    // Interval (in seconds) at which the system checks for
    findings
    // and reports them to Splunk
    pub report_interval: u64,

    // An Api key for virus total.
    pub virus_total_api_key: String,

    // A list of directories to monitor
    pub directory_to_watch: Vec<String>,

    // The path to the machine learning module module
    pub module_path: String,

    // Path to the scaled data.
    pub module_scale_path: String,
}
```

These are the configurations user has to provide for the system to function correctly. A default implementation has been applied nevertheless its quite important for the user to provide them as informations such the module_path, module_scale_path, token and api_key. Can not be generated by default

5.4.6. Static Analysis Implementation

Static Analysis focus on one thing is verifying processes executables against VirusTotal malware database. This can help with any kind of malware as VirusTotal covers a lot of malware beside Ransomware.

Main Structure for Static Analysis.

```
pub struct Signature {
    algorithm_types: AlgorithmType, // Either MD5/SHA256
    client: Client, // Http client used to send requests to
    VirusTotal
    alert_manager: AlertManager, // A copy from the AlertManager
    discussed earlier to send alerts
    stop: Arc<AtomicBool>, // an Atomic boolean help to stop the
    thread on termination.
    system: System, // An instance of System object provided by
    `sysinfo` to gather processes
}
```

Major functions used in Static Analysis.

```
pub async fn verify_file_signature(&mut self) {
    for proc in Process::get_processes(&self.system) {
        if let Some(path) = proc.exec {
            let file_bytes = self.read_file(&path).await;
            match file_bytes {
                Some(bytes) => {
                    let hash =
self.calculate_signature(bytes).await;
                    let res =
self.make_virus_total_req(hash).await;
                    if res {
self.alert_manager.send_alert(AlertingInfo(proc.pid, 100.0));
                    } else {
                        continue;
                    }
                }
                None => {
```

```

        warn!("No data to process or hash")
    }
}
} else {
    warn!("No path to find hash for.");
}
}
}

```

This is the main function used to gather processes and get the executable file as bytes to generate a signature and check them.

```

async fn make_virus_total_req(&self, hash: String) -> bool {
    let path = format!("https://www.virustotal.com/api/v3/
files/{}", hash);
    let req = self.client.get(path).send().await;
    match req {
        Ok(res) => res.status() == StatusCode::OK,
        Err(err) => {
            warn!("Error making request to virustotal {}",
err);
            false
        }
    }
}

```

This other function used to send requests to VirusTotal along with the signature of the file if the file match and exist in the database. The Status Code will be 200 OK if not it will return NOT FOUND 404.

5.4.7. Machine Learning Integration

The `ModuleFeature` class bridges the gap between collected data and ML inference:

Prepare Function: This function scales the raw data to match the training data distribution. Scaling is essential because the Random Forest model was trained on normalized data. The implementation applies the same scaling factors used during training.

Order Function: Ensures features are arranged in the exact order expected by the model. The ONNX model expects features in a specific sequence matching the training dataset. Any mismatch would result in incorrect predictions.

ONNX Runtime Integration: The `ort` crate is used to load and run the trained Random Forest model. The process:

1. Load the ONNX model file at startup
2. For each prediction request, create an input tensor with the feature vector
3. Order and arrange the vector
4. Scale the vector
5. Pass the vector to the **ML** module.
6. Extract the prediction probability.
7. If higher than threshold send alert.
8. Other wise discard.

Challenges Integration With Rust: One major challenge when trying to integrate the module with **Rust** were that **Rust** dose not work with **skLearn** so a conversion must be in place to turn it into another format that can be used within. One idea was to convert it into **ONNX** format.

However the module after conversion suffers from a different metrics than the **skLearn** version of the module. And after searching it turns out that integrating the scaled data into the exported **ONNX** module was the cause of metrics problem so obviously the solution was to include a non-scaled data into the exported module and when trying feed the module the data scale is needed before the module can make accurate predications.

5.4.8. Alert Management

Alert Management class is responsible for sending requests to **Splunk** “Only”. As other Structure used to represent the actual request.

```
pub struct AlertManager {  
    pub client: Client,  
    splunk_path: Url,  
    pub min_alert: Severity, // a minimum severity to alert  
    index_name: String,  
}
```

Here everything is self explanatory. Except index_name this is related on how **Splunk** arranges the events. The following structure is used to represent the request shape.

```
struct SplunkRequestOption {  
    /// The event timestamp. Splunk uses this to index the event  
    correctly.  
    /// If omitted, Splunk will use the ingestion time.  
    time: u64,  
  
    /// Source of the event, usually the system or application  
    generating it (e.g., "soar_tool").  
    source: String,  
  
    /// Splunk sourcetype. Classifies the kind of event for  
    indexing and field extraction.  
    /// Example: "security:alert", "web:access", etc.  
    sourcetype: String,  
  
    /// Index in Splunk where this event will be stored (e.g.,  
    "security", "main").  
    /// Helps organize events by type or team.  
    index: String,  
  
    /// The actual event payload containing detailed information.  
    event: Event,  
}
```

Here everything is commented and self explanatory as well. Except the Event this is actually another structure used and shown as following.

```
struct Event {
    /// Unique identifier for this event.
    /// Helps prevent duplicates and allows referencing the event
    later.
    id: String,

    /// Type of the event (e.g., "phishing_email",
    "malware_detected").
    /// `[serde(rename = "type")]` ensures it is serialized as
    "type" in JSON,
    /// because `type` is a reserved Rust keyword.
    #[serde(rename = "type")]
    type_event: String,

    /// Severity level of the event (e.g., "low", "medium", "high",
    "critical").
    /// Useful for dashboards, alerts, and filtering in Splunk.
    severity: String,

    /// Human-readable description of the event.
    /// Provides context for SOC analysts or for auditing.
    description: String,

    /// Action that was taken automatically or manually (e.g.,
    "quarantined", "blocked", "ignored").
    action_taken: String,

    /// Tags for categorizing or filtering events (e.g.,
    ["phishing", "soar", "malware"]).
    /// Useful for dashboards, searches, and field extraction.
    tags: Vec<String>,
}
```

These structures help to generate understandable message for **Splunk** to arrange and index.

5.5. Machine Learning Model Training

The machine learning component is implemented in Python using scikit-learn and related libraries.

5.5.1. Dataset Preparation

As discussed in Section 4.5, the final dataset combines two sources:

1. eBPFangel dataset: Provides system call patterns
2. RanSMAP dataset: Provides entropy measurements

Data Loading: The datasets are loaded using pandas from CSV files. And the label is being dropped as it does not matter for the machine learning itself rather used for classification.

```
data = pd.read_csv("Path_to_csv.csv")
X = data.drop("LABEL", axis=1)
y = data["LABEL"]
```

Data Combination: Basically a number of **Entropy** values are extracted from the **RanSMAP** dataset and added to the data from **eBPFangel** that are labeled as **Ransoms** typically and generally having a high **entropy** denote either **zipping** operation or **encryption** operation.

Oversampling: The original eBPFangel dataset was imbalanced with far more benign samples than malware samples. A library called **imblearn** was used to apply oversampling techniques, creating synthetic malware samples to balance the dataset. This ensures the model doesn't develop a bias toward predicting benign behavior.

```
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train_scaled,
y_train)
```

Split & Scale Data The following snippet is just a stander machine learning process.

```
# splitting
X_train, X_test, y_train, y_test = train_test_split(
    X_noisy, y_mod, test_size=0.2, random_state=42, stratify=y_mod
)

# Scalling.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Random Forest Training

```
rf = RandomForestClassifier(random_state=42, max_depth=7,
min_samples_leaf=3)
rf.fit(X_train_res, y_train_res)
```


5.5.2. Model Export to ONNX

After training, the scikit-learn model was exported to ONNX (Kim et al., 2022) format using `skl2onnx`. This allows the model to be used in the Rust application via the ONNX Runtime. One minor issue **ONNX** kinda suffer from which is that it does not process data as **float 64** rather a **float 32** this is not a big of a deal but its valid to point it out.

```
from skl2onnx import to_onnx

# Convert to numpy array with explicit shape
X_sample = X[:,1].values.astype(np.float32)

onx = to_onnx(rf, X_sample)
with open("rf_iris.onnx", "wb") as f:
    f.write(onx.SerializeToString())
```

5.6. Deployment Configuration

5.6.1. System Requirements

The deployed system requires: As for the kernel version it has been tested on Linux 6.17.9-arch1-1 and any version that support **eBPF** should be able to run the program smoothly. Also a **Root** privileges is require to load the **eBPF** module and interact with it.

5.6.2. Configuration File

Example structure:

```
min_severity_to_report = "Medium"

splunk_path = "http://localhost:8088"

splunk_token = "Token here..."

index_name = "security"

report_interval = 10

virus_total_api_key = "API_KEY_HERE"

directory_to_watch = ["/tmp/", "/root/", "/var/", "/dev/", "/etc/"]

module_path = "$HOME/Desktop/FYP/project/learn_bpf/
extract_sysCalls_to_map/testing/ml/rf_iris.onnx"

module_scale_path = "$HOME/Desktop/FYP/project/learn_bpf/
extract_sysCalls_to_map/testing/ml/scaler.json"

hash_algorithm = "MD5"
```

5.6.3. Installation Steps

1. Install dependencies (Rust, Python for training)
2. Compile eBPF programs
3. Build Rust application
4. Add your configuration file in **/root/.crab_shield**

5. Run with root privileges `sudo RUST_LOG=info cargo run`. (you can discard the `RUST_LOG=info` part but if you want logs you can add it)

5.7. Summary

This chapter covered the complete implementation of the ransomware detection system, from development environment setup through testing and deployment. The implementation successfully translates the design from Chapter 4 into working code, utilizing eBPF for kernel-level monitoring, Rust for efficient user-space processing, and machine learning for intelligent detection.

The key achievements include:

- A working eBPF program that monitors file operations with minimal overhead
- Integration of static and dynamic analysis approaches
- Real-time detection using machine learning with 97.58% accuracy
- Automated alerting through Splunk integration

6. Testing and Validation

6.1. Application Testing

Application test cases is more into utilizing the great Rust modularity for testing. The code snippet below is a sample of how testing is done.

```
#[cfg(test)]
mod test {
    use std::{
        path::PathBuf,
        str::FromStr,
        sync::{Arc, atomic::AtomicBool},
    };

    use crate::{
        alert::handler_alert::AlertManager,
        config_parser::parser::Config,
        singature::sig::Signature,
    };

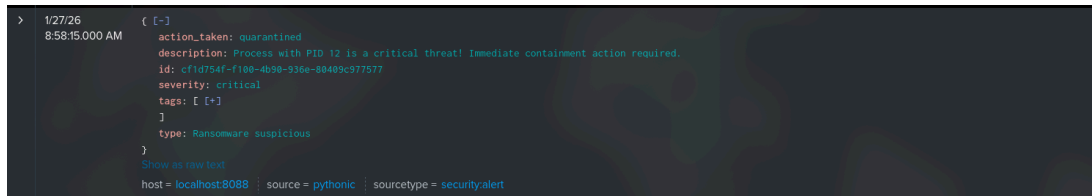
    #[tokio::test]
    async fn test_signature() {
        let path_config = PathBuf::from_str("/
root/.crab_sheild.toml").unwrap();
        let config = Config::new(&path_config).await;
        let mut alert = AlertManager::new(&config);
        let automic = Arc::new(AtomicBool::new(false));
        let mut sig = Signature::new(
            super::AlgorithmType::MD5,
            config.virus_total_api_key,
            alert.clone(),
            &automic,
        );
        let malware_path =
            PathBuf::from_str("/home/pythonic/Desktop/FYP/
MalwareSample/sample.exe").unwrap();
        let file_bytes = sig.read_file(&malware_path).await;
        let md5 = sig.calculate_md5(file_bytes.unwrap());
        let res = sig.make_virus_total_req(md5).await;
```

```
        alert
            .send_alert(crate::machine_learning_module::AlertingInfo(12,
100.0))
            .await;
        assert!(res, "Alert did not sent");
    }
}
```

Utilizing this we can know that the application can logically work correct.

6.2. Use Case Testing

The primary use case of the application is to run it and be able to send alerts based on your configurations. The Figure 12 shows the application indexes into **Splunk** and every other data included in the event.



```
> 1/27/26 8:58:15.000 AM { [-]
  action_taken: quarantined
  description: Process with PID 12 is a critical threat! Immediate containment action required.
  id: cf1d754f-f180-4b50-936e-80489c377577
  severity: critical
  tags: [ [+]]
  type: Ransomware suspicious
}
Show as raw text
host = localhost8088 source = pythonic sourcetype = security:alert
```

Figure 12: Use case testing

6.3. Malware Testing.

Testing actual malware and running them is risky and requires additional hardware to safely run Ransomwares. You can achieve that using virtual machines nevertheless the harm that can be caused for network access preventing me from testing an actual malware. Due to limited resources.

7. Conclusion

This project help the ongoing hype of detecting and preventing Ransomware attacks. This project may seems as a simple step toward the goal nevertheless am very happy with the achievements that i was able to do and this is not end of it this project going to be continued developed until it reach a state where it can be actually effective in protecting others from the harm that Ransomware groups are causing.

7.1. Summary of Achievements

The primary achievement of this project is the demonstrated ability to commit to a long-term project and continuously learn despite initial unfamiliarity with the technologies involved.

Second, utilizing eBPF to enable kernel level monitoring proved to be very effective as shown in this project. However, it can be challenging at the beginning due to the lack of resources to learn from. One of the best available resources is (Rice, 2023). This resource was instrumental in achieving the eBPF implementation shown in this project.

Third, the strong implication of Rust in the field of security is clearly demonstrated in the project along with its ease of use. While expertise in Rust is still developing, the modularity Rust provides and the simple basic building blocks it offers helped achieve a great system architecture in a very efficient manner. C++, on the other hand, can be very handy, especially when working with eBPF. Both remain excellent choices.

Fourth, utilizing machine learning was not a goal at the start of the project. Moving forward, integrating ML into the project proved interesting. Initially, machine learning concepts were unfamiliar. Nevertheless, using machine learning in integration with security represents a very interesting research area as it opens new windows of capabilities that can be achieved.

Lastly, the overall system design was successful, though improvements are planned for the near future, including support for Windows OS. Until then, the project will remain mainly Linux focused.

7.2. Lessons Learned

During these processes, several challenges created insightful opportunities. The strict safety requirements mandated by the eBPF verifier challenge necessitated creative approaches for path matching and data collection. The strict programming environment of kernel programming also provided a valuable lesson on its own—kernel programming is unforgiving of error-prone code bugs.

The value of machine learning lies in the pipeline. First and foremost, the machine learning pipeline revealed the importance of the quality of the dataset. Since the Table 1 dataset is imbalanced, techniques for over-sampling are needed. Using a combined dataset from various data sources—firstly system calls from the (Willers & Philippart, 2024) dataset and secondly entropy from the (Hirano & Kobayashi, 2025) dataset—proved helpful.

However, during the process of converting the scikit-learn model, some complications that related to the handling of data types were encountered. Additionally, the inability to add the scaling factors during the export process led to the addition of a scaling pipeline within the Rust code. This experience highlighted the difficulties involved in the process of bridging the gap between machine learning models and production environments.

From the test results, it was evident that the limitations of the behavioral detection process existed. Although the process proved effective as a defense mechanism for ransomware attacks, some new forms of cyber-attacks might be missed by the system until an update is made to the machine learning model. This further supports the idea, as presented by an expert during the questionnaire process, that the world of cybersecurity will continue to be a cat-and-mouse game as part of an ongoing arms race.

7.3. Limitations and Future Considerations

The main limitation of this project is related to the consumption of resources. It is to be noted that despite the implementation of asynchronous operation, high CPU utilization may still be encountered. The major limitation encountered is related to the Ring Buffer used for the transmission of events, which requires the use of a while loop.

Additionally, the lack of support from the Windows OS is a limitation that can be addressed in future.

References

- Alqahtani, A., & Sheldon, F. T. (2022). A Survey of Crypto Ransomware Attack Detection Methodologies: An Evolving Outlook. *Sensors*, 22(5). <https://doi.org/10.3390/s22051837>
- Arch Linux Project. (2026,). *Arch Linux Official Website*.
- Aslan, Ö. A., & Samet, R. (2020). A Comprehensive Review on Malware Detection Approaches. *IEEE Access*, 8, 6249–6271. <https://doi.org/10.1109/ACCESS.2019.2963724>
- Breus, A., Lasker, H., Antonov, P., Gattuso, L., & Andersen, V. (2024). *Real-Time Ransomware Detection Using Behavioral Entropy Analysis for Secure File Systems*. <https://doi.org/10.36227/techrxiv.173091498.80588045/v1>
- Cen, M., Jiang, F., Qin, X., Jiang, Q., & Doss, R. (2024). Ransomware early detection: A survey. *Computer Networks*, 239, 110138. <https://doi.org/https://doi.org/10.1016/j.comnet.2023.110138>
- Git Community. (2026,). *Git Official Website*.
- Hirano, M., & Kobayashi, R. (2025). RanSMAP: Open dataset of Ransomware Storage and Memory Access Patterns for creating deep learning based ransomware detectors. *Computers & Security*, 150, 104202. <https://doi.org/https://doi.org/10.1016/j.cose.2024.104202>
- Kapoor, A., Gupta, A., Gupta, R., Tanwar, S., Sharma, G., & Davidson, I. E. (2022). Ransomware Detection, Avoidance, and Mitigation Scheme: A Review and Future Directions. *Sustainability*, 14(1). <https://doi.org/10.3390/su14010008>
- Kim, S. Y., Lee, J., Kim, C. H., Lee, W. J., & Kim, S. W. (2022). Extending the ONNX Runtime Framework for the Processing-in-Memory Execution. *2022 International Conference on Electronics, Information, And Communication (ICEIC)*, , 1–4. <https://doi.org/10.1109/ICEIC54506.2022.9748444>
- Kiyol, S., Franchini, D., Moreno, E., Kowalski, R., Fernandez, W., & Milosevic, A. (2024). *Ransomware Detection Using LSTM Networks and File Entropy Analysis: A Sequence-Based Approach*. <https://doi.org/10.22541/au.172841694.49477944/v1>
- Manoharan, S., & poonkuzhali.S. (2020). *A novel approach to detect IoT malware by system calls using Deep learning techniques*. <https://doi.org/10.1109/ICITIIT49094.2020.9071531>
- McIntosh, T., Jang-Jaccard, J., Watters, P., & Susnjak, T. (2019). *The Inadequacy of Entropy-Based Ransomware Detection* (pp. 181–189). https://doi.org/10.1007/978-3-030-36802-9_20
- Mohammed, R., Rawashdeh, J., & Abdullah, M. (2020). Machine Learning with Oversampling and Undersampling Techniques: Overview Study and Experimental Results. *2020 11th International Conference on Information and*

- Communication Systems (ICICS)*, , 243–248. <https://doi.org/10.1109/ICICS49469.2020.239556>
- Neovim Project. (2026,). *Neovim Official Website*.
- Random Forest Algorithm Overview. (2024). *Babylonian Journal of Machine Learning*, 2024, 69–79. <https://doi.org/10.58496/BJML/2024/007>
- Rice, L. (2023). *Learning eBPF: Programming the Linux Kernel for Enhanced Observability, Networking, and Security*. O'Reilly Media.
- Svet, L., Brightwell, A., Wildflower, A., & Marshwood, C. (2025,). *Unveiling Zero-Space Detection: A Novel Framework for Autonomous Ransomware Identification in High-Velocity Environments* (p.). <https://doi.org/10.48550/arXiv.2501.12811>
- Typst Project. (2026,). *Typst Official Website*.
- Virustotal. (n.d.). *Virustotal home page*. <https://www.virustotal.com/gui/home/upload>
- Willers, M., & Philippart, T. (2024,). *ebpfangel: Ransomware Detection using Machine Learning with eBPF for Linux*.
- Wu, Y., & Chang, Y. (2024). *Ransomware detection on linux using machine learning with random forest algorithm*. <https://doi.org/10.36227/techrxiv.171778770.06550236/v1>

